

1. IntroduçãoManual do Usuário

MyAI Artificial Intelligence

O Futuro da Automação Inteligente

1. Introdução

O futuro da automação não é apenas código. É inteligência organizada, programada e executada no momento certo.

O **MyAI Artificial Intelligence** foi criado para transformar a maneira como sistemas inteligentes são construídos. Uma plataforma onde tarefas são agendadas com precisão, processos são divididos em microsserviços e eventos coordenam toda a execução de forma inteligente.

Cada automação se torna um ecossistema vivo de processos, capaz de executar rotinas complexas exatamente na data e hora definidas. No coração do MyAI está a união de duas forças poderosas:

- **A flexibilidade e o ecossistema do Python**
- **A performance e estabilidade do Pascal**

Com uma abordagem **NoCode** e **LowCode**, o MyAI permite que desenvolvedores e empresas criem sistemas completos de automação sem complexidade desnecessária — mantendo sempre acesso a todo o poder do Python quando necessário.

O resultado é uma nova forma de construir automação:

- ✂ Agendada
- ✂ Orientada por eventos
- ✂ Baseada em microsserviços
- ✂ Potencializada por Python
- ✂ Acelerada por Pascal

2. Guia de Instalação2. Guia de Instalação

Siga os passos abaixo para configurar o ambiente em sua máquina. O MyAI é um aplicativo nativo para **Windows**.

[!IMPORTANT] Pré-requisitos:

1. Para realizar a extração dos arquivos do sistema (passo 5), você precisará do aplicativo **WinRAR** instalado.
2. O MyAI utiliza Python para seus agentes de inteligência artificial. O instalador automatizado está incluso no pacote.

Passo a Passo:

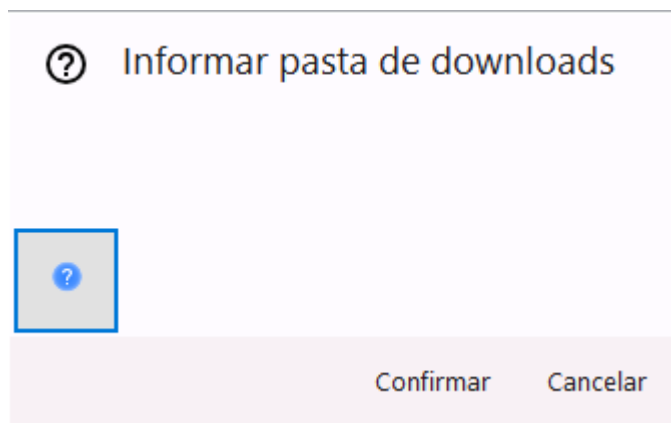
1. **Criação do Diretório:** No seu explorador de arquivos, crie uma pasta diretamente na unidade **C:** com o nome **MyAI** (Exemplo: **C:\MyAI**).
2. **Download do Projeto:** Baixe os arquivos necessários diretamente do repositório oficial no GitHub: <https://github.com/DataNews-Tecnologia-Ltda/MyAI>
3. **Organização:** Copie o arquivo baixado (**MyAI-main.zip**) da sua pasta de *Downloads* para a pasta criada em **C:\MyAI**
4. **Extração do Repositório:** Extraia o conteúdo do arquivo **MyAI-main.zip** dentro da própria pasta **C:\MyAI**
5. **Extração do Executável:** * Acesse o caminho: **C:\MyAI\MyAI-main\aplicativo**.
 - Localize o arquivo **MyAI.rar**, clique com o botão direito e selecione "**Extrair aqui**" (usando o WinRAR).
6. **Configuração do Ambiente Python (Modo Desenvolvedor):**
 - Antes de abrir o aplicativo pela primeira vez, você deve preparar o ambiente Python.
 - Navegue até: **C:\MyAI\MyAI-main\aplicativo**
 - Execute o arquivo **instalar_python.bat** em modo administrador.
 - Este script instalará o Python e todos os pacotes básicos necessários para o funcionamento das automações.
7. **Inicialização:** * Agora, execute o arquivo: **C:\MyAI\MyAI-main\aplicativo\MyAI.exe**
 - *Dica: Recomendamos clicar com o botão direito no executável e selecionar "Criar atalho" para fixá-lo em sua Área de Trabalho*

3. Configuração Inicial3. configuração Inicial (Primeiro Acesso)

Ao abrir o **MyAI.exe** pela primeira vez, o sistema solicitará a definição dos caminhos de diretório para que as automações funcionem corretamente.

3.1 Informar Pasta de Downloads

Esta configuração permite que o MyAI saiba onde localizar arquivos baixados ou onde salvar resultados de automações de web scraping e processamento de dados.



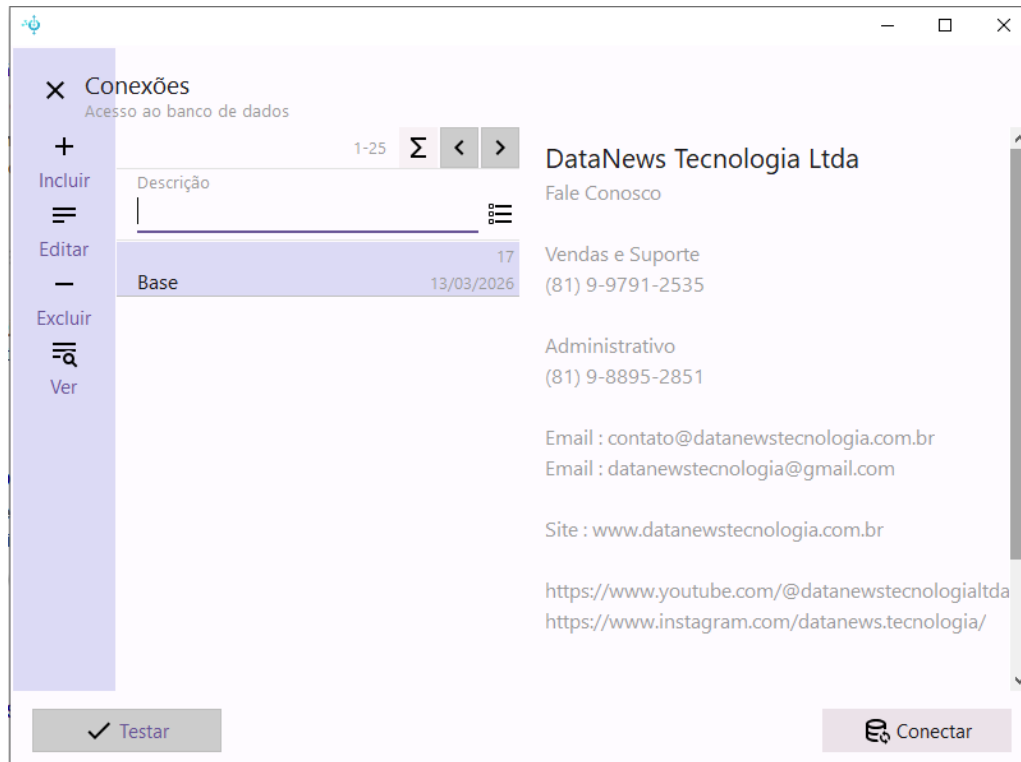
Passo a Passo:

1. Na janela flutuante com o título **"Informar pasta de downloads"**, clique no botão cinza que contém o ícone de **interrogação (?)**.
2. Uma janela do explorador de arquivos será aberta.
3. Navegue até o local padrão do Windows: geralmente localizado em **Este Computador > Downloads**.
4. Selecione a pasta e clique em **Confirmar**.

4. Conexão com a Base de Dados4.

Conexão com a Base de Dados

Após configurar os caminhos de diretório, o sistema exibirá a tela de **Conexões**. Este passo é essencial para vincular o MyAI ao seu banco de dados de automações, lembrando que já está configurado.



Passo a Passo para Conectar:

1. **Seleção:** Clique sobre o item com a descrição "**Base**". Note que, ao selecionar, a linha ficará destacada.
2. **Ação de Conectar:** Com a base selecionada, clique no botão **Conectar**, localizado no canto inferior direito da tela.

Observações Importantes:

- **Suporte:** Caso de suporte, os contatos da **DataNews Tecnologia** (Telefones, E-mail e Redes Sociais) estão visíveis no lado direito da tela para suporte imediato.

5. Autenticação e Controle de Acesso5.

Autenticação e Controle de Acesso

Após estabelecer a conexão com a base de dados, o sistema apresentará a tela de login. O MyAI utiliza um sistema de hierarquia onde o Administrador controla as permissões de novos usuários.

5.1 Primeiro Acesso (Administrador)

Para o primeiro acesso ao sistema, utilize as credenciais padrão de administrador:

1. **Usuário:** Digite **aitools** e pressione a tecla **Enter**.
2. **Senha:** Digite **aitools**.
3. **Ação:** Clique no botão **Acessar**.

Segurança: Por questões de segurança, recomendamos que a senha do administrador seja alterada imediatamente após o primeiro login através das configurações de perfil dentro do app.

5.2 Cadastro de Novos Usuários

O MyAI permite que múltiplos colaboradores utilizem a plataforma, seguindo o fluxo abaixo:

1. **Solicitação:** O novo colaborador deve clicar em **"Inscreva-se agora >"**
2. **Cadastro:** Preencher os dados solicitados para criar a conta.
3. **Bloqueio de Segurança:** Por padrão, todo novo usuário cadastrado inicia com o status **Bloqueado**, O **Administrador** do sistema deve acessar a área de gestão de usuários dentro do aplicativo para **desbloquear** o novo perfil.

6. Termos de Uso e Ativação da Licença

Após o login, o sistema apresentará a tela de licenciamento. Para utilizar o **MyAI**, o usuário deve aceitar as condições contratuais e realizar a ativação da assinatura.



6.1 Aceite de Contrato

Antes de prosseguir para o pagamento, é obrigatório validar os termos de direito de uso.

- Leia atentamente o contrato exibido na aba "Contrato".
- Marque a caixa de seleção: **"Li e aceito os termos, marque para poder prosseguir"**.

6.2 Configuração de Pagamento e Assinatura

Na aba **Pagamento**, você poderá personalizar seu plano de acordo com a necessidade da sua empresa.

Passo a Passo para Ativação:

1. **Seleção do Plano (Produto):** Escolha o período de validade da licença (30, 90, 180 ou 360 dias).
2. **Identificação Fiscal:** Informe obrigatoriamente seu **CPF/CNPJ** e seu **E-mail**. Esses dados são fundamentais para a emissão posterior da sua **NFSe**.
3. **Serviços Adicionais (Opcionais):**
 - **Suporte Técnico:** Mediante pagamento, você terá direito a auxílio técnico especializado na instalação.
 - **Treinamento:** Mediante pagamento, você terá direito a 30 minutos de treinamento remoto para operar a plataforma.
4. **Geração do PIX:**
 - Clique em **Gerar PIX**.
 - O sistema exibirá o **QR Code** e a chave **PIX (Copia e Cola)**.
5. **Liberação do App:**
 - Após realizar o pagamento no seu banco, volte ao MyAI e clique em **Consultar Pagamento**.
 - Assim que o sistema confirmar a transação, o acesso total às funcionalidades do aplicativo será liberado automaticamente.

7. Suporte Técnico e Acesso Remoto7.

Suporte Técnico e Acesso Remoto

Como o processo de instalação e treinamento é realizado de forma **100% remota**, o MyAI exige o uso de ferramentas de compartilhamento de tela para que nossa equipe técnica possa atuar em sua máquina.

7.1 Ferramentas de Acesso

Para solicitar suporte técnico ou realizar o treinamento contratado, você deve ter instalado um dos seguintes softwares:

- **RustDesk** (Recomendado)
- **AnyDesk**

7.2 Como Proceder:

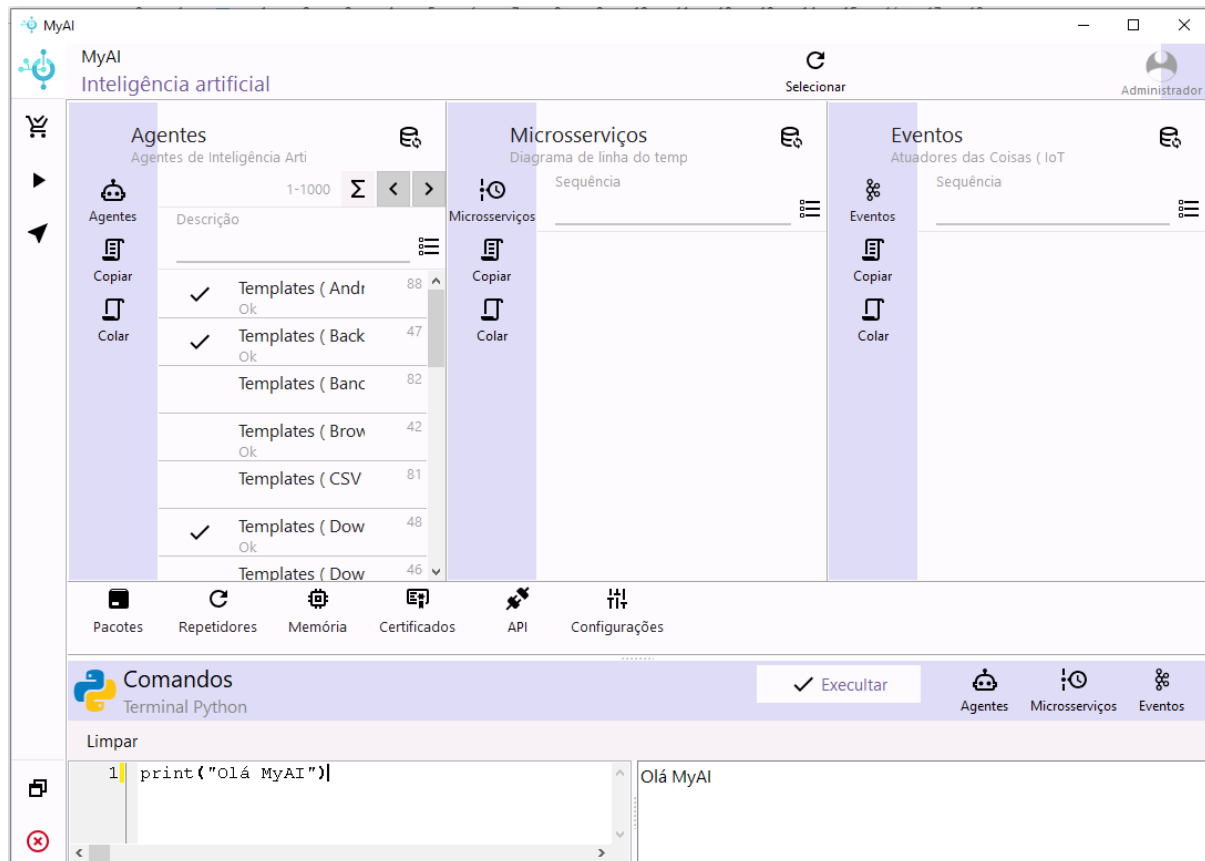
1. **Download:** Caso não possua nenhum deles, baixe e instale a versão oficial diretamente dos sites dos desenvolvedores.
2. **Preparação:** Antes de entrar em contato com o suporte da **DataNews Tecnologia**, abra o aplicativo de acesso remoto escolhido.
3. **Identificação:** Tenha em mãos o **ID** e a **Senha** gerados pelo software para fornecer ao técnico responsável.

[!TIP] Manter essas ferramentas instaladas agiliza o diagnóstico de eventuais problemas e garante que seu SUPORTE seja aproveitado ao máximo, sem perder tempo com downloads de última hora.

8. Dashboard Básico

(Visão Geral)

A Dashboard do MyAI é o centro de comando onde a arquitetura modular do sistema se torna visível. A interface é dividida em três pilares principais de execução, além de ferramentas de suporte e um terminal integrado.



8.1 Os Três Pilares da Automação

A parte superior da Dashboard é organizada em colunas que representam o fluxo da sua inteligência artificial:

1. **Agentes:** Onde você gerencia as entidades inteligentes (Agentes de Inteligência Artificial) que tomam decisões ou executam tarefas específicas.
2. **Microserviços:** Onde os processos são divididos em pequenos diagramas de linha do tempo. Aqui você define a sequência de ações que compõem uma rotina.
3. **Eventos:** Onde estão os atuadores das coisas (IoT). Os eventos são os gatilhos que coordenam quando e como cada ação deve ser disparada (baseado em data, hora ou condições).

9. Agentes9. Agentes (Agentes de Inteligência Artificial)

Os **Agentes** no MyAI são o cérebro das suas automações. Eles funcionam como "trabalhadores incansáveis" que executam scripts Python em momentos pré-determinados, garantindo que suas tarefas críticas nunca sejam esquecidas.



9.1 Campos Informativos (Automáticos)

Alguns campos são gerados pelo sistema para controle de auditoria e status:

- **Id**: Identificador único do registro no banco de dados.
- **Data Bloqueio**: Preenchido automaticamente.
- **Data e Hora Execução**: Registra o exato momento que o Agente rodou modo Standby

9.2 Configuração de Prioridade

Define a ordem de importância caso duas tarefas coincidam no mesmo horário.

- 1 - Alta / 2 - Média / 3 - Baixa

9.3 Configuração de Período (A Inteligência do Tempo)

Dependendo da sua escolha no campo **Período**, o MyAI habilitará campos específicos para detalhamento:

- **Diário** (Informe apenas a **Hora**)
- **Semanal** (**Dia da Semana** (Segunda a Domingo) e a **Hora**)
- **Mensal** (Informe o **Dia do Mês** (1 a 31) e a **Hora**)
- **Anual** (Selecione o **Mês**, o **Dia** e a **Hora** para rotinas de longo prazo.)
- **Avulso** (Não possuem repetição automática programada (execução manual))

Atenção: Para que qualquer Agente ou tarefa agendada seja executada sem intervenção humana, o MyAI precisa estar em modo ativa (Standby).

9.3 Drive (Selenium)

Evitar erro de incompatibilidade de outras tarefas sendo processadas ao mesmo tempo, informar um nome sem espaços e sem ponto. exemplo = drive_agents_001

9.5 Bloqueio de Segurança

O interruptor **Bloqueado (Sim/Não)** permite pausar um Agente instantaneamente sem precisar excluir suas configurações ou scripts. Útil para manutenções rápidas nos microsserviços associados.

10. Microserviços10. Microserviços

(Diagrama de linha do tempo)

O módulo de Microserviços é o coração operacional do MyAI. Ele permite fragmentar uma automação complexa em uma sequência lógica de pequenas tarefas (o Diagrama de Linha do Tempo), facilitando a manutenção e o reaproveitamento de processos.



10.1 Campos Informativos (Automáticos)

Alguns campos são gerados pelo sistema para controle de auditoria e status:

- **Id**: Identificador único do registro no banco de dados.
- **Data Bloqueio**: Preenchido automaticamente.

10.2 Sequência

A ordem numérica de execução. (Ex: 001, 002, 003...).

10.3 Descrição

Nome descritivo da etapa.

10.4 Ação

É uma classificação, natureza técnica do microserviços.

Aplicativo : Utilizado para abrir softwares instalados no Windows. Deve-se informar o caminho completo do executável no campo **Caminho** (Ex: **.exe**)

Controle : Funções de teclado e mouse

Internet : **Selenium**. Utilizado para web scraping e navegação automatizada em browsers via Python.

Arquivo : **Data Science**. Utilizado para manipulação de arquivos (leitura, escrita, conversão)

Fantasma : (*Atualmente desativado*)

Se : Condição para acesso ao microserviço

Nome (Variável): Informe o nome de uma variável previamente criada na **Memória**.

Expressão: Escolha o operador lógico (**>**, **<**, **=**, **<>**, **C=** para Contém, etc.).

Conteúdo: O valor que será comparado. ex. 2 ou

`{{ReaderVariable[</48/></sysTotalPage/>]}}`

Laço : Inicia um ciclo de repetição, **não colocar eventos**.

Fim Laço : Define o ponto de retorno para o início do ciclo, **não colocar eventos**, preencher (Nome, Expressão e Conteúdo) da variável, **Salvar** (informar uma variável a ser atribuída com a sequência atual do laço)

11. Eventos11. Eventos (Atuadores das Coisas - IoT)

O módulo de Eventos é responsável por **executar ações** granulares que compõem os microsserviços. É aqui que você configura a interação direta com o sistema operacional, navegadores, ciência de dados (Data Science), aprendizado de máquina (Machine Learning), análise de dados (Data Analytics : Power BI)



10.1 Campos Informativos (Automáticos)

Alguns campos são gerados pelo sistema para controle de auditoria e status:

- **Id**: Identificador único do registro no banco de dados.
- **Data Bloqueio**: Preenchido automaticamente.

10.2 Sequência

A ordem numérica de execução. (Ex: 001, 002, 003...).

10.3 Condição para acesso ao evento

Nome (Variável): Informe o nome de uma variável previamente criada na **Memória**.

Expressão: Escolha o operador lógico (>, <, =, <>, C= para Contém, etc.).

Conteúdo: O valor que será comparado.

10.4 Acesso ao Ecosystema: Clique no ícone de engrenagem (botão de teste) para abrir a janela de configuração do Ecosystema



10.4.1 Classificação de Ações e Motores Python + Pascal

Dentro da aba **Ações** do Ecosystema, o MyAI utiliza bibliotecas específicas do Python para cada tarefa;

Mouse : Controle de cursor, cliques e localização de imagens pacote pyautogui

Keyboard : Simulação de digitação e comandos de teclado pacote pyautogui

Message : Criação de janelas de alerta, confirmação e mensagens pacote pyautogui

Script : Execução direta de comandos e scripts personalizados comandos nativo do python

Browser : Automação completa de navegadores (Chrome, Firefox, Edge) pacote selenium

Function : Integração de funções internas do Pascal + Python pacotes (pyperclip, os, sys)

File : Manipulação física de arquivos no diretório do sistema pacotes (sys, os)

Data : Processamento de Ciência de Dados (Data Science) pacotes (pandas, sys)

11.1. Eventos

10.4.2 Funções Específicas por Ação

Cada ação habilita um conjunto de funções.

10.4.3 Interface de Desenvolvimento de Eventos

A janela de configuração possui campos e ferramentas para garantir que o evento funcione antes de ser salvo:

1. **Descrição:** Texto informativo sobre a finalidade do evento.
2. **Parâmetros:** Campos dinâmicos que se habilitam conforme a função escolhida para configuração de coordenadas, textos ou caminhos.
3. **Script (Python):** Exibe ou permite inserir os comandos Python brutos.
4. **Retorno (Python)#:** Mostra o log de saída ou erros após o teste.

Botões de Ferramental

1. **Limpar:** Apaga o histórico de comandos executados no teste.
2. **Rastrear:** Copia todo o histórico de comandos para a área de transferência para depuração, você pode colocar no IDLE (python).
3. **Ver Script:** Permite visualizar o código final que o MyAI montou automaticamente.
4. **Teclado:** Abre uma lista auxiliar com os nomes técnicos das teclas para facilitar o uso na ação Keyboard.
5. **Testar:** Executa o evento isoladamente para validação imediata.
6. **Salvar:** Grava as definições no Ecossistema (**lembre-se de salvar também no painel de Eventos principal após fechar esta janela**).

Abas de Especialização

Além da aba principal de **Ações**, o Ecossistema conta com:

- **Navegador:** Configurações exclusivas para quando a ação for **Browser**.
- **Dados:** Painel dedicado à ação **Data** (**atualmente em desenvolvimento para integração total com Data Science**).

12.1. Eventos : Mouse : position12.

Dicionário de Funções: Ação Mouse

A ação **Mouse** utiliza o motor `pyautogui.move` para interagir com a interface visual do Windows. O MyAI facilita a configuração dessas funções através de atalhos inteligentes de captura de tela.

12.1 Função: `position`

Esta função é a base para a maioria das interações. Ela move o cursor do mouse para um ponto específico da tela.

1. **Descrição:** Moverá o cursor do mouse para as coordenadas inteiras X e Y.
2. **Parâmetros:** Recebe as coordenadas no formato `</X/></Y/>`.
 - *Exemplo:* `</151/></169/>` move o mouse para a posição horizontal 151 e vertical 169.

Recurso Inteligente: Captura Automática ("M")

Para facilitar o preenchimento dos parâmetros, o MyAI exibe a instrução no topo da tela: **"Posicione o mouse e tecla 'M'".**

1. Posiciona no campo parâmetros e tecla (Home) no seu teclado, mova o mouse fisicamente até o botão ou área desejada na sua tela.
2. Pressione a tecla **"M"** no seu teclado físico.
3. O MyAI lerá a posição exata do cursor naquele instante e preencherá automaticamente o campo **Parâmetros** com as coordenadas corretas.

12.2. Eventos : Mouse : click12. Dicionário de Funções: Ação Mouse

A ação **Mouse** utiliza o motor `pyautogui.click()`.

12.2 Função: **click**

Esta função é utilizada para interagir diretamente com elementos da interface, como botões, menus e links.

3. **Descrição:** Executa o clique do botão esquerdo do mouse na posição atual em que o cursor se encontra.

12.3. Eventos : Mouse : doubleclick¹².

Dicionário de Funções: Ação Mouse

A ação **Mouse** utiliza o motor `pyautogui.doubleClick()`.

12.3 Função: `doubleclick`

Esta função é utilizada para interagir diretamente com elementos da interface, como botões, menus e links.

4. **Descrição:** Executa dois cliques rápidos com o botão esquerdo do mouse na posição atual em que o cursor se encontra.

12.4. Eventos : Mouse : clickimage12.

Dicionário de Funções: Ação Mouse

A ação **Mouse** utiliza o motor `pyautogui.position() + pyautogui.screenshot() + pyautogui.click()` para interagir com a interfaces dinâmicas do Windows.

12.4 Função: **clickimage**

Esta função detecta imagens, onde os botões podem mudar de posição na tela. Em vez de depender de coordenadas fixas, o MyAI "enxerga" o elemento visual desejado.

5. **Descrição:** Localiza uma imagem específica na tela e executa o clique com o botão esquerdo do mouse sobre ela.
6. **Parâmetros:** Nome do arquivo de imagem `</img.png/>`.

Recurso de Captura de Imagem (Teclas "1" e "2")

Para configurar esta função, o MyAI oferece um sistema de captura integrado indicado na parte superior da tela: **"Msg: Capturar imagem tecla '1' e '2'"**.

1. **Tecla "1":** Define o ponto inicial (canto superior esquerdo) da área da imagem que você deseja transformar em um "botão" para a automação.
2. **Tecla "2":** Define o ponto final (canto inferior direito).
3. O sistema recorta essa área, salva o arquivo e preenche automaticamente o campo **Parâmetros** com a referência da nova imagem.

12.5. Eventos : Mouse :

doubleclickimage12. Dicionário de Funções:

Ação Mouse

A ação **Mouse** utiliza o motor `pyautogui.position()` + `pyautogui.screenshot()` + `pyautogui.click()` para interagir com a interfaces dinâmicas do Windows.

12.5 Função: `doubleclickimage`

Esta função detecta imagens, onde os botões podem mudar de posição na tela. Em vez de depender de coordenadas fixas, o MyAI "enxerga" o elemento visual desejado.

7. **Descrição:** Localiza uma imagem específica na tela e executa dois cliques rápidos com o botão esquerdo do mouse sobre ela.
8. **Parâmetros:** Nome do arquivo de imagem `</img.png/>`.

Recurso de Captura de Imagem (Teclas "1" e "2")

Para configurar esta função, o MyAI oferece um sistema de captura integrado indicado na parte superior da tela: **"Msg: Capturar imagem tecla '1' e '2'"**.

4. **Tecla "1":** Define o ponto inicial (canto superior esquerdo) da área da imagem que você deseja transformar em um "botão" para a automação.
5. **Tecla "2":** Define o ponto final (canto inferior direito).
6. O sistema recorta essa área, salva o arquivo e preenche automaticamente o campo **Parâmetros** com a referência da nova imagem.

12.6. Eventos : Mouse :

locateonscreen12. Dicionário de Funções: Ação Mouse

A ação **Mouse** utiliza o motor `pyautogui.position() + pyautogui.screenshot() + pyautogui.click()` para interagir com a interfaces dinâmicas do Windows.

12.6 Função: **locateonscreen**

Essa é uma função de **segurança crítica**. Ela atua como um "sentinela" no seu fluxo: se o elemento visual esperado não estiver lá, o **MyAI** interrompe tudo para evitar cliques em locais errados ou erros em cascata.

9. **Descrição:** Busca uma imagem específica na tela. Se a imagem **não for encontrada**, o sistema gera um erro fatal e **paralisa imediatamente** todos os processos e o agente em execução.
10. **Parâmetros:** Nome do arquivo de imagem `</img.png/>`.

Recurso de Captura de Imagem (Teclas "1" e "2")

Para configurar esta função, o MyAI oferece um sistema de captura integrado indicado na parte superior da tela: **"Msg: Capturar imagem tecla '1' e '2'"**.

7. **Tecla "1":** Define o ponto inicial (canto superior esquerdo) da área da imagem que você deseja transformar em um "botão" para a automação.
8. **Tecla "2":** Define o ponto final (canto inferior direito).
9. O sistema recorta essa área, salva o arquivo e preenche automaticamente o campo **Parâmetros** com a referência da nova imagem.

12.7. Eventos : Mouse : positionclick12.

Dicionário de Funções: Ação Mouse

A ação **Mouse** utiliza o motor `pyautogui.click` para interagir com a interface do Windows.

12.7 Função: `positionclick`

A função `positionclick` é a combinação técnica perfeita entre movimento e ação. Ela elimina falhas comuns em automações onde o clique é disparado antes do mouse terminar o deslocamento.

11. **Descrição:** Realiza o movimento do cursor para uma coordenada específica (X e Y) e, imediatamente após a chegada, executa um clique com o botão esquerdo.
12. **Parâmetros:** Utiliza o formato padrão de coordenadas `</X/></Y/>`

Recurso Inteligente: Captura Automática ("M")

Para facilitar o preenchimento dos parâmetros, o MyAI exibe a instrução no topo da tela: **"Posicione o mouse e tecle 'M'".**

4. Posiciona no campo parâmetros e tecle (Home) no seu teclado, mova o mouse fisicamente até o botão ou área desejada na sua tela.
5. Pressione a tecla **"M"** no seu teclado físico.
6. O MyAI lerá a posição exata do cursor naquele instante e preencherá automaticamente o campo **Parâmetros** com as coordenadas corretas.

12.8. Eventos : Mouse :

positiondoubleclick12. Dicionário de Funções: Ação Mouse

A ação **Mouse** utiliza o motor `pyautogui.click` para interagir com a interface do Windows.

12.8 Função: `positiondoubleclick`

A função `positiondoubleclick` é a combinação técnica perfeita entre movimento e ação. Ela elimina falhas comuns em automações onde o clique é disparado antes do mouse terminar o deslocamento.

13. **Descrição:** Realiza o movimento do cursor para uma coordenada específica (X e Y) e, imediatamente após a chegada, executa um clique com o botão esquerdo.

14. **Parâmetros:** Utiliza o formato padrão de coordenadas `</X/></Y/>`

Recurso Inteligente: Captura Automática ("M")

Para facilitar o preenchimento dos parâmetros, o MyAI exibe a instrução no topo da tela: **"Posicione o mouse e tecla 'M'".**

7. Posiciona no campo parâmetros e tecla (Home) no seu teclado, mova o mouse fisicamente até o botão ou área desejada na sua tela.
8. Pressione a tecla **"M"** no seu teclado físico.
9. O MyAI lerá a posição exata do cursor naquele instante e preencherá automaticamente o campo **Parâmetros** com as coordenadas corretas.

12.9. Eventos : Mouse :

positionclickwrite12. Dicionário de Funções: Ação Mouse

A ação **Mouse** utiliza o motor `pyautogui.click` + `pyautogui.write` para interagir com o Windows.

12.9 Função: `positionclickwrite`

Esta função representa a junção tripla das operações de posicionamento, foco e entrada de dados. Ela é ideal para preenchimento de campos de texto, telas de login ou qualquer interface que exija uma interação sequencial rápida.

15. **Descrição:** Escreve em uma determinada coordenada específica (X e Y).

16. **Parâmetros:** Utiliza o formato padrão de coordenadas `</X/></Y/>`

Recurso Inteligente: Captura Automática ("M")

Para facilitar o preenchimento dos parâmetros, o MyAI exibe a instrução no topo da tela: **"Posicione o mouse e tecla 'M'".**

10. Posiciona no campo parâmetros e tecla (Home) no seu teclado, mova o mouse fisicamente até o botão ou área desejada na sua tela.
11. Pressione a tecla **"M"** no seu teclado físico.
12. O MyAI lerá a posição exata do cursor naquele instante e preencherá automaticamente o campo **Parâmetros** com as coordenadas corretas.

12.10. Eventos : Mouse :

positionclickclearwrite12. Dicionário de Funções: Ação Mouse

A ação **Mouse** utiliza o motor `pyautogui.click` + `pyautogui.write` para interagir com o Windows.

12.10 Função: `positionclickclearwrite`

Esta função representa a junção tripla das operações de posicionamento, foco, `apaga` e entrada de dados. Ela é ideal para preenchimento de campos de texto, telas de login ou qualquer interface que exija uma interação sequencial rápida.

17. **Descrição:** Escreve em uma determinada coordenada específica (X e Y).

18. **Parâmetros:** Utiliza o formato padrão de coordenadas `</X/></Y/>`

Recurso Inteligente: Captura Automática ("M")

Para facilitar o preenchimento dos parâmetros, o MyAI exibe a instrução no topo da tela: **"Posicione o mouse e tecla 'M'".**

13. Posiciona no campo parâmetros e tecla (Home) no seu teclado, mova o mouse fisicamente até o botão ou área desejada na sua tela.
14. Pressione a tecla **"M"** no seu teclado físico.
15. O MyAI lerá a posição exata do cursor naquele instante e preencherá automaticamente o campo **Parâmetros** com as coordenadas corretas.

12.11. Eventos : Mouse : clickright¹².

Dicionário de Funções: Ação Mouse

A ação **Mouse** utiliza o motor `pyautogui.click()`.

12.11 Função: **clickright**

Esta função é utilizada para interagir diretamente com elementos da interface, como botões, menus e links.

19. **Descrição:** Executa o clique do botão direito do mouse na posição atual em que o cursor se encontra.

12.12. Eventos : Mouse :

positionclickright12. Dicionário de Funções:

Ação Mouse

A ação **Mouse** utiliza o motor `pyautogui.click` para interagir com a interface do Windows.

12.12 Função: `positionclickright`

A função `positionclickright` é a combinação técnica perfeita entre movimento e ação. Ela elimina falhas comuns em automações onde o clique é disparado antes do mouse terminar o deslocamento.

20. **Descrição:** Realiza o movimento do cursor para uma coordenada específica (X e Y) e, imediatamente após a chegada, executa um clique com o botão direito.

21. **Parâmetros:** Utiliza o formato padrão de coordenadas `</X/></Y/>`

Recurso Inteligente: Captura Automática ("M")

Para facilitar o preenchimento dos parâmetros, o MyAI exibe a instrução no topo da tela: **"Posicione o mouse e tecla 'M'".**

16. Posiciona no campo parâmetros e tecla (Home) no seu teclado, mova o mouse fisicamente até o botão ou área desejada na sua tela.
17. Pressione a tecla **"M"** no seu teclado físico.
18. O MyAI lerá a posição exata do cursor naquele instante e preencherá automaticamente o campo **Parâmetros** com as coordenadas corretas.

12.13. Eventos : Mouse :

existonscreen12. Dicionário de Funções: Ação Mouse

A ação **Mouse** utiliza o motor `pyautogui.click` para interagir com a interface do Windows.

12.13 Função: **existonscreen**

Esta função de nível avançado vincula a localização de uma imagem à execução ou status de um Agente específico, permitindo uma automação baseada em estados visuais e persistência de memória.

22. **Descrição:** Busca uma imagem na tela e, dependendo do sucesso da localização, interage com um Agente e gerência variáveis de memória para decidir o próximo passo do fluxo.

23. **Parâmetros;**

- a. : Imagem a pesquisar na tela
- b. : O número identificador (ID#) do Agente atual
- c. : O nome da variável de memória que receberá o status da operação
- d. : O conteúdo ou comando que deve ser gravado na variável **seja encontrada**
- e. : O conteúdo ou comando que deve ser gravado na variável **não seja encontrada**

Recurso de Captura de Imagem (Teclas "1" e "2")

Para configurar esta função, o MyAI oferece um sistema de captura integrado indicado na parte superior da tela: **"Msg: Capturar imagem tecla '1' e '2'"**.

- 10. **Tecla "1":** Define o ponto inicial (canto superior esquerdo) da área da imagem que você deseja transformar em um "botão" para a automação.
- 11. **Tecla "2":** Define o ponto final (canto inferior direito).
- 12. O sistema recorta essa área, salva o arquivo e preenche automaticamente o campo **Parâmetros** com a referência da nova imagem.

12.14. Eventos : Mouse : clicktexto12.

Dicionário de Funções: Ação Mouse

A ação **Mouse** utiliza o motor **time + pyautogui + pytesseract + cv2 + numpy** para interagir com a interface do Windows.

12.13 Função: **clicktexto**

Esta função realiza um fluxo complexo em milissegundos: tira um print da tela, converte os pixels em texto legível, localiza a palavra desejada e clica nas coordenadas geográficas desse texto e persiste na memória.

24. **Descrição:** Captura a tela, converte para texto via OCR, pesquisa o termo desejado (sem diferenciar maiúsculas de minúsculas) e executa o clique no local encontrado

25. Parâmetros;

- a. : Texto a pesquisar na tela
- b. : O número identificador (ID#) do Agente atual
- c. : O nome da variável de memória que receberá o status da operação
- d. : O conteúdo ou comando que deve ser gravado na variável **seja encontrada**
- e. : O conteúdo ou comando que deve ser gravado na variável **não seja encontrada**.
- f. : Correção X, valor numérico para deslocar o clique
- g. : Correção Y, valor numérico para deslocar o clique

12.15. Eventos : Mouse :

doubleclicktexto12. Dicionário de Funções: Ação Mouse

A ação **Mouse** utiliza o motor `time + pyautogui + pytesseract + cv2 + numpy` para interagir com a interface do Windows.

12.13 Função: `doubleclicktexto`

Esta função realiza um fluxo complexo em milissegundos: tira um print da tela, converte os pixels em texto legível, localiza a palavra desejada e clica duas vezes nas coordenadas geográficas desse texto e persiste na memória.

26. **Descrição:** Captura a tela, converte para texto via OCR, pesquisa o termo desejado (sem diferenciar maiúsculas de minúsculas) e executa o clique no local encontrado

27. Parâmetros;

- a. : Texto a pesquisar na tela
- b. : O número identificador (ID#) do Agente atual
- c. : O nome da variável de memória que receberá o status da operação
- d. : O conteúdo ou comando que deve ser gravado na variável **seja encontrada**
- e. : O conteúdo ou comando que deve ser gravado na variável **não seja encontrada**.
- f. : Correção X, valor numérico para deslocar o clique
- g. : Correção Y, valor numérico para deslocar o clique

13.1. Eventos : Script : time

13. Dicionário de Funções: Ação Script

A ação **Script** é utilizada para executar comandos diretos no interpretador Python do ecossistema motor `time.sleep(5)`. Aqui, você não está limitado a funções pré-definidas; você tem o poder total da linguagem Python para criar cálculos, manipulações de texto e, principalmente, pausas estratégicas.

13.1 Função: `time`

Esta é uma das funções mais vitais para a estabilidade de qualquer robô. Ela introduz uma pausa controlada na execução do código.

- 28. **Descrição:** Faz com que o Agente aguarde um número específico de segundos antes de prosseguir para a próxima linha de comando ou para o próximo evento.
- 29. **Parâmetros:** Recebe o tempo de espera no formato `</segundos/>`
 - *Exemplo:* `</5/>` fará o sistema parar por exatos 5 segundos.

Por que o `time` é indispensável?

- 1. **Deteção de Bots:** Softwares e sites modernos monitoram a velocidade de interação. Cliques humanos não são instantâneos. Usar `time` com valores aleatórios (ex: 1.5s, 2s) ajuda a humanizar o comportamento do seu Agente.
- 2. **Sincronização de Processamento:** Muitas vezes, um clique abre uma janela que demora alguns segundos para carregar. Sem a função `time`, o próximo comando (como digitar ou clicar) falharia porque o elemento ainda não estaria na tela.
- 3. **Economia de CPU:** Evita que o sistema entre em "loop infinito" de verificação, consumindo recursos desnecessários enquanto aguarda uma resposta externa.

13.2. Eventos : Script : script13. Dicionário de Funções: Ação script

O campo **Script (Python)** permite que desenvolvedores e usuários avançados digitem códigos Python puro para serem executados instantaneamente pelo motor do ecossistema. **Isso elimina qualquer limitação criativa, permitindo que você crie lógicas personalizadas que não existem nos menus pré-definidos.**

13.2 Função: **script**

Uma das maiores vantagens é poder manipular as **Variáveis de Memória** do MyAI diretamente via código.

30. **Descrição:** Executa códigos Python.

O usuário tem o controle total do motor Python!

13.3. Eventos : Script : java

13.3. Dicionário de Funções: Ação java

O campo **Script (Python)** muda sua propriedade para **Script (Java)**, permite a execução de código externo, especificamente classes Java, integrando o poder dessa linguagem ao ecossistema de eventos.

13.3 Função: java

Esta função gerencia o ciclo de vida de um arquivo Java, desde a compilação até a execução, de forma automatizada.

31. **Descrição:** Executa uma classe Java. O sistema verifica a existência do arquivo binário e decide se precisa compilar o código fonte antes de rodar.
32. **Parâmetros:** O primeiro parâmetro deve ser o **nome do arquivo fonte (sem extensão) e da classe Java**

Regra de Modificação

É fundamental lembrar que o Java não re compila automaticamente se houver mudanças no código fonte após a primeira geração do binário:

- Para aplicar alterações: Se você modificar o código dentro do arquivo `.java`, deve apagar manualmente o arquivo `.class` antigo.
- Isso força o MyAI a identificar a ausência do binário e realizar uma nova compilação com as modificações recentes.

Requisito de Sistema

Para que esta função opere corretamente, é necessário que o **JDK (Java Development Kit)** esteja instalado e configurado nas variáveis de ambiente do Windows, permitindo que os comandos `java` e `javac` sejam acessíveis pelo terminal, em configurações deve colocar o path do **JavaC**.

14.1. Eventos : Keyboard : write14.

Dicionário de Funções: Ação Keyboard

A ação **Keyboard** utiliza o motor `pyautogui.write` para simular a digitação de textos e o acionamento de teclas especiais. Ela é fundamental para preencher formulários, enviar mensagens ou executar atalhos do sistema.

14.1 Função: `write`

Esta função é utilizada para digitar sequências de caracteres em campos de texto que já possuem o foco (cursor ativo).

- 33. **Descrição:** Escreve um texto de forma sequencial.
- 34. **Parâmetros:** Recebe o conteúdo a ser digitado entre as tags `</ >`
 - *Exemplo:* `</Ola Mundo!/>` resultará na digitação literal da frase

Uso de Expressões Dinâmicas

O MyAI possui inteligência para converter variáveis de sistema em tempo real durante a digitação.

- **{{DATE}}:** Esta expressão é automaticamente substituída pela data atual no formato `yyyy_mm_dd`.
 - *Exemplo de Parâmetro:* `</Ola Mundo!, a data de hoje é {{DATE}}/>`
 - *Resultado (em 17/03/2026):* `Ola Mundo!, a data de hoje é 2026_03_17`

14.2. Eventos : Keyboard : press14.

Dicionário de Funções: Ação Keyboard

A ação **press** utiliza o motor `pyautogui.press`

14.2 Função: **press**

É utilizada para comandos de controle e navegação. Ela simula o ato de pressionar e soltar uma tecla específica do teclado.

35. **Descrição:** Executa o acionamento de uma tecla funcional.

36. **Parâmetros:** Recebe o nome técnico da tecla entre as tags `</ />`

○ *Exemplo:* `</enter/>`, `</tab/>`, `</esc/>`, `</f5/>`

O Botão Teclado (Assistente de Comandos)

Como o Python (via `pyautogui`) exige nomes específicos para as teclas (ex: utilizar "capslock" em vez de "Fixa"), o MyAI inclui um facilitador visual:

1. Ao clicar no botão **Teclado** no painel de ações, uma janela auxiliar é exibida com a lista completa de teclas suportadas.
2. **Facilidade:** Você não precisa decorar os nomes. Basta consultar a lista para garantir que o parâmetro inserido seja exatamente o que o motor Python espera receber.

14.3. Eventos : Keyboard : hotkey¹⁴.

Dicionário de Funções: Ação Keyboard

A função **hotkey** utiliza o motor `pyautogui.hotkey`, é a ferramenta definitiva para executar atalhos combinados. Ela simula o pressionamento de várias teclas simultaneamente, algo essencial para operações rápidas de sistema que, de outra forma, exigiriam vários cliques de mouse.

14.3 Função: **hotkey**

Esta função é utilizada para disparar combinações de teclas de atalho (Ex: Ctrl+C, Alt+Tab, Win+R).

- 37. **Descrição:** Executa uma combinação de duas ou mais teclas simultâneas.
- 38. **Parâmetros:** Recebe os nomes das teclas separados por vírgula dentro dos marcadores `</ />`
 - *Exemplo:* `</ctrl/></esc/>` ou `</ctrl,c/>`

O Botão Teclado (Assistente de Comandos)

Como o Python (via `pyautogui`) exige nomes específicos para as teclas (ex: utilizar "capslock" em vez de "Fixa"), o MyAI inclui um facilitador visual:

- 3. Ao clicar no botão **Teclado** no painel de ações, uma janela auxiliar é exibida com a lista completa de teclas suportadas.
- 4. **Facilidade:** Você não precisa decorar os nomes. Basta consultar a lista para garantir que o parâmetro inserido seja exatamente o que o motor Python espera receber.

14.4. Eventos : Keyboard : copy14.

Dicionário de Funções: Ação Keyboard

A função **copy** utiliza o motor [pyperclip.copy](#), essa é uma solução técnica para lidar com as limitações de codificação de caracteres (UTF-8, ASCII) e layouts de teclado variados. Quando o motor de digitação direta falha com símbolos especiais ou acentos, o uso da Área de Transferência (Clipboard) garante 100% de fidelidade ao dado original.

14.4 Função: **copy**

A função **copy** é o método mais seguro para inserir textos complexos, senhas com caracteres especiais ou grandes blocos de informação em um campo de destino. Em vez de "digitar" tecla por tecla, o MyAI injeta o conteúdo diretamente na memória de transferência do Windows.

39. **Descrição:** Copia o conteúdo definido nos parâmetros para a área de transferência.

40. **Parâmetros:** Recebe o texto literal ou a variável entre os marcadores `</ />`.

○ *Exemplo:* `</Senha@Complexa#2026_!/>`

Por que usar o **copy** em vez do **write**?

1. **Caracteres Especiais:** Símbolos como ç, ã, §, ou emojis podem não ser reconhecidos pelo driver de teclado em certos idiomas. O **copy** ignora o teclado e lida com o texto puro.
2. **Velocidade:** Enquanto o **write** digita caractere por caractere (com intervalos), o **copy** disponibiliza o texto instantaneamente.
3. **Segurança de Dados:** Evita erros de "teclas presas" ou atrasos de rede que podem embaralhar a digitação em sistemas remotos (como o acesso via RDP ou TeamViewer).

15.1. Eventos : Message : msgalert15.

Dicionário de Funções: Ação Message

A ação **Message** utiliza o motor `pyautogui.alert`, diferente das ações de teclado e mouse que interagem silenciosamente com o sistema, as funções de **Message** criam janelas pop-up (caixas de diálogo) na interface do Windows.

15.1 Função: `msgalert`

Esta função é utilizada para exibir uma notificação simples que interrompe a execução do robô até que o usuário clique em um botão de confirmação.

41. **Descrição:** Exibe uma mensagem de alerta personalizada na tela.

42. **Parâmetros;**

- a. O corpo da mensagem que será lido pelo usuário.
- b. O título que aparecerá na barra superior da janela.
 - o O rótulo do botão de confirmação (ex: "OK", "Entendi", "Continuar").

Benefícios na Automação

- **Pausa Inteligente:** O robô aguarda a interação humana, garantindo que o próximo passo só ocorra após a ciência do operador.
- **Personalização:** Você pode usar variáveis de memória nos parâmetros para exibir resultados dinâmicos.

15.2. Eventos : Message : confirm15.

Dicionário de Funções: Ação Message

A ação **Message** utiliza o motor [pyautogui.confirm](#).

15.2 Função: **confirm**

Esta função **confirm** cria uma janela de diálogo interativa com múltiplas opções. Ela é utilizada quando o fluxo de automação precisa de uma resposta específica do operador para decidir qual caminho seguir.

43. **Descrição:** Exibe uma mensagem de confirmação que permite ao usuário escolher entre um e até três botões diferentes

44. **Parâmetros;**

- c. O conteúdo informativo ou pergunta da janela
- d. O título na barra superior da janela.
- e. Define os rótulo do primeiro botão
- f. Define os rótulo do segundo botão
- g. Define os rótulo do terceiro botão

A Variável de Memória **{{MSGResult}}** no próximo evento e Python **sysResult** como uma variável

Sempre que uma função **confirm** é executada, o rótulo do botão que foi clicado pelo operador é armazenado instantaneamente na variável de memória **{{MSGResult}}**.

- **Como funciona:** Esta variável funciona como uma "ponte" entre a interface visual e a lógica do script.
- **Persistência:** O valor de **{{MSGResult}}** permanece guardado até que uma nova mensagem de confirmação seja exibida e clicada, sobrescrevendo o valor anterior.

Vantagens Técnicas

1. **Rastreabilidade:** No campo **Retorno (Python)#**, você verá a confirmação do clique (ex: **Resultado MSG: Sim**), facilitando o debug do seu fluxo.
2. **Dinamicidade:** Permite criar automações semi-assistidas, onde o robô faz o trabalho pesado, mas para e pergunta ao humano em pontos críticos de decisão.

15.3. Eventos : Message : prompt15.

Dicionário de Funções: Ação Message

A ação **Message** utiliza o motor `pyautogui.prompt`.

15.3 Função: `prompt`

Esta função exibe uma caixa de diálogo com um campo de entrada de texto. É a ferramenta ideal para capturar informações que mudam a cada execução, como um número de protocolo, uma chave de acesso ou um nome de arquivo específico.

45. **Descrição:** Exibe uma janela solicitando que o usuário digite uma informação.

46. **Parâmetros;**

- h. A instrução do que o usuário deve digitar (ex: "Digite o CPF do cliente").
- i. O título na barra superior da janela.
- j. Um texto que já virá preenchido no campo

A Variável de Memória `{{MSGResult}}` e Python `sysResult`

Sempre que uma função `prompt` é executada, Captura do Conteúdo Digitado.

1. **Variável de Memória:** O texto digitado é salvo na variável `{{MSGResult}}`.
2. **Ambiente Python:** O valor também fica disponível através da variável `sysResult`, permitindo manipulação direta via script.

O rótulo do botão que foi clicado pelo operador é armazenado instantaneamente na variável de memória `{{MSGResult}}`.

- **Como funciona:** Esta variável funciona como uma "ponte" entre a interface visual e a lógica do script.
- **Persistência:** O valor de `{{MSGResult}}` permanece guardado até que uma nova mensagem de confirmação seja exibida e clicada, sobrescrevendo o valor anterior.

16.1. Eventos : Function :

copyclipboardrightHTML16. Dicionário de Funções: Ação Function

Esta é uma das funções mais avançadas e especializadas da categoria **Function** (Python + Pascal). Ela não é apenas um comando, mas um "combo" de automação projetado especificamente para extrair o código-fonte (HTML) de elementos em navegadores como o Google Chrome, utilizando as ferramentas de desenvolvedor (DevTools).

16.1 Função: **copyclipboardrightHTML**

Esta função automatiza uma sequência complexa de comandos de teclado e mouse para capturar o HTML interno de um elemento específico na tela. **É ideal para quando o raspagem de dados (scraping) via Selenium não é possível ou quando se precisa do conteúdo exato do DOM rapidamente.**

47. **Descrição:** Acessa o modo desenvolvedor, navega pelos menus de contexto e copia o conteúdo estruturado do elemento para a área de transferência do Windows.
48. **Parâmetros:** Recebe as coordenadas `</X/></Y/>` de onde o processo deve iniciar (onde o clique direito será dado).

Quando utilizar?

- Para extrair dados de sites que possuem **proteção contra robôs**.
- Quando você precisa capturar um valor que só aparece no "Inspeccionar Elemento" e não está visível no texto da página.
- Para capturar tabelas complexas que perdem a formatação ao serem copiadas normalmente.

16.2. Eventos : Function :

locatereplaceHTML 16. Dicionário de Funções: Ação Function

A ação **Function** é para pesquisar na área de transferência do windows, com a localização de HTML, seu robô agora consegue "ler" e "entender" o contexto das páginas web!

16.2 Função: locatereplaceHTML

Esta função atua como um filtro inteligente para a área de transferência. Ela é frequentemente utilizada logo após a `copyclipboardrightHTML` para **validar se o robô está na página correta** ou se um elemento específico foi carregado com sucesso.

49. **Descrição:** Realiza uma pesquisa de texto dentro do conteúdo que está atualmente na área de transferência (clipboard) e armazena o resultado em uma variável de memória

50. **Parâmetros;**

- a. (ID) Identificador do agente
- b. Texto a pesquisar, no código HTML pesquisar qualquer coisa
- c. Nome da variável de memória
- d. Caso encontre conteúdo a gravar na variável
- e. Caso não encontre conteúdo a gravar na variável

Aplicação Prática no Navegador (Browser)

A maior utilidade desta função é a **confirmação de página**. Em automações web, nem sempre o carregamento é instantâneo. Ao usar `locatereplaceHTML`:

1. O robô pesquisa por um texto único da página (ex: "Bem-vindo, João" ou um ID de tabela).
2. Se o texto for encontrado, a variável de memória é atualizada, permitindo que o fluxo prossiga com segurança.
3. Isso evita que o robô tente clicar em botões que ainda não existem na tela, aumentando a estabilidade do evento.

16.3. Eventos : Function :

updatevariableHTML

16. Dicionário de Funções: Ação Function

A ação **Function** é para pesquisar na área de transferência do windows, e faz filtro de informações.

16.3 Função: **updatevariableHTML**

Enquanto as funções anteriores localizam ou copiam o conteúdo, a **updatevariableHTML** é a responsável por filtrar o que está na área de transferência e salvar o dado limpo em uma variável de memória específica do MyAI.

51. **Descrição:** Salva o conteúdo presente na área de transferência em uma variável de memória, permitindo limitar a leitura e extrair apenas dados numéricos

52. Parâmetros;

- f. (ID) Identificador do agente
- g. Limite de caracteres que serão lidos da área de transferência
- h. Utilize o caractere "**N**" se desejar que o sistema extraia apenas números
- i. Nome da variável de memória

Exemplo Prático: Captura de Quantidade

Se um site exibe o texto `<h1>200 documentos encontrados</h1>` e você deseja apenas o número:

1. Use `copyclipboardrightHTML` para copiar o cabeçalho.
2. Configure `updatevariableHTML` com o filtro `</N/>`.
3. A variável `<?sysTesting?>` receberá apenas o valor `200`.

16.4. Eventos : Function :

updatelinkvariableHTML

16. Dicionário de Funções: Ação Function

A ação **Function** para capturar e armazenar o endereço (URL) de um link específico de uma página web para uso posterior, o MyAI utiliza uma função dedicada que interage diretamente com a estrutura do navegador.

16.4 Função: **updatelinkvariableHTML**

Esta função é uma variação especializada na extração de HTML. Em vez de copiar o código interno de um elemento, ela foca especificamente em capturar o atributo **href** (o link de destino) de um elemento clicável que está na área de transferência.

53. **Descrição:** Salva o conteúdo URL na área de transferência em uma variável de memória.

54. **Parâmetros;**

- j. (ID) Identificador do agente
- k. Nome da variável de memória

Vantagem Estratégica

Esta função é vital para automações de "crawler", onde o robô precisa identificar links de documentos (como PDFs de notas fiscais ou boletos) que **são gerados dinamicamente e cujos nomes de arquivo mudam a cada acesso**.

16.5. Eventos : Function :

pasteclipboardrightHTML16. Dicionário de Funções: Ação Function

A ação **Function** é uma função de manipulação direta do **DOM (Document Object Model)** do navegador. Enquanto as funções anteriores serviam para "ler" a página, a **pasteclipboardrightHTML** serve para "escrever" ou alterar visualmente o conteúdo de um site em tempo real, utilizando as ferramentas de desenvolvedor (DevTools).

16.5 Função: **pasteclipboardrightHTML**

Esta função localiza um texto específico dentro do código HTML da área de transferência e o substitui por um novo conteúdo, forçando essa alteração diretamente na interface do navegador Chrome.

55. **Descrição:** Acessa o modo desenvolvedor, pesquisa um texto no código do elemento e, se encontrado, substitui pelo novo texto definido, atualizando a visualização da página.

56. **Parâmetros;**

- l. (X) Coordenada
- m. (Y) Coordenada
- n. O texto original que você deseja localizar para substituir.
- o. O novo conteúdo que será injetado no lugar do antigo.

Casos de Uso e Utilidade

- **Correção de Dados em Tela:** Útil para ajustar informações visualmente antes de gerar um print screen ou PDF comprobante.
- **Testes de Interface:** Permite simular como um campo se comporta com diferentes tamanhos de texto ou valores sem precisar alterar o banco de dados real do site.
- **Automação de Sistemas Rígidos:** Pode ser usada para "limpar" campos ou textos que impedem o prosseguimento de uma macro de interface.

[!WARNING] **Atenção:** Esta alteração é apenas **visual e local**. Ela não altera os dados no servidor do site. Se você atualizar a página (F5), a modificação desaparecerá e o texto original retornará.

16.6. Eventos : Function :

savevariable16. Dicionário de Funções: Ação

Function

A ação **Function** é uma função de capacidade de **substituição dinâmica**.

16.6 Função: **savevariable**

Esta função é utilizada para gravar ou atualizar valores dentro das variáveis de memória do sistema. Ela permite que um dado capturado em uma etapa (como um número de protocolo ou um nome) seja armazenado e reutilizado por qualquer outro Microserviço ou Evento posteriormente ou setar o início de um loop (laço).

57. **Descrição:** Salva um conteúdo específico em uma variável de memória vinculada a um Agente.

58. **Parâmetros;**

- p. (ID) Identificador do agente
- q. O nome da variável onde o dado será guardado
- r. O valor real que você deseja salvar. Pode ser um texto fixo, um número ou o resultado de uma operação anterior.

Dinâmica de Substituição

- **Exemplo:** Se você quiser salvar uma frase completa, pode usar:
</37/></sysTexto/></O status do processo é : {{sysStatus}}/>. O MyAI processará o valor de {{sysStatus}} e salvará o resultado final na variável de destino sysTexto.

16.7. Eventos : Function :

locatevariable16. Dicionário de Funções: Ação Function

A ação **Function** é uma função de capacidade de **pesquisa e substituição**.

16.7 Função: **locatevariable**

Esta função é uma ferramenta poderosa de "Pesquisar e Substituir" que atua diretamente sobre as variáveis de memória já existentes. Ela permite validar conteúdos e gerar novas informações baseadas em condições de texto.

59. **Descrição:** Pesquisa um texto ou o conteúdo de uma variável e, caso encontre uma correspondência, realiza a substituição por um novo valor, salvando o resultado final em uma variável de destino.

60. **Parâmetros;**

- s. (ID) Identificador do agente
- t. O texto fixo que servirá de base para a busca.
- u. A variável de memória cujo conteúdo será analisado.
- v. O novo texto ou valor que entrará no lugar do que foi localizado.
- w. A variável que receberá o resultado final da operação.

Exemplo de Lógica Aplicada

Imagine que você tem uma variável chamada **<?sysTexto?>** com o conteúdo "Aguardando Certificado".

- Se você configurar o parâmetro de pesquisa para "Certificado" e o de substituição para "OK", a função pode gerar uma nova variável **<?status_final?>** com o conteúdo "Aguardando OK".

16.8. Eventos : Function :

unionvariable16. Dicionário de Funções: Ação Function

A ação **Function** é uma função de capacidade ideal para a composição de informações complexas a partir de partes menores.

16.8 Função: **unionvariable**

Esta função permite a concatenação (junção) de duas variáveis distintas em uma terceira variável de destino. Ela é essencial para montar caminhos de arquivos, frases completas ou chaves de busca compostas.

61. **Descrição:** Junta o conteúdo de duas variáveis de memória e salva o resultado em uma nova variável.

62. Parâmetros;

- x. (ID) Identificador do agente
- y. O primeiro conteúdo da junção (variável de memória).
- z. O segundo conteúdo da junção (variável de memória).
- aa. O nome da variável que receberá a combinação das duas anteriores.

Exemplo de Lógica Aplicada

É uma otimização para a estrutura lógica de "A + B = C", garantindo que a integridade dos dados originais seja mantida enquanto a nova informação é gerada.

16.9. Eventos : Function : fileexists16.

Dicionário de Funções: Ação Function

A ação **Function** na **Ação Fileexists**, que lida diretamente com a segurança e a integridade dos arquivos no sistema. A primeira função essencial é a de verificação, que serve como um "guardião" do fluxo, são críticas porque garantem que o robô não tente processar algo que não existe

16.9 Função: fileexists

Esta função é um comando de validação preventiva. Ela verifica se um arquivo específico está presente no diretório indicado antes de permitir que o Agente prossiga para o próximo evento.

63. **Descrição:** Verifica a existência de um arquivo. Caso o arquivo **não seja encontrado**, o sistema gera um erro crítico que **paralisa a execução do Agente** imediatamente por segurança.

64. **Parâmetros:** Recebe o caminho completo do arquivo entre os marcadores `</ />`.

- *Exemplo:* `</C:\Automacao\Relatorios\vendas_hoje.csv/>`.

Uso com Variáveis Dinâmicas

Você pode tornar essa verificação muito mais poderosa utilizando as variáveis que aprendemos anteriormente. Se você acabou de baixar um arquivo com a data de hoje:

- *Parâmetro:* `</C:\Downloads\relatorio_{{DATE}}.pdf/>`
- Isso garante que o robô valide exatamente o arquivo do dia corrente.

16.10. Eventos : Function :

resultpython16. Dicionário de Funções: Ação Function

A ação **Function** na **Ação resultpython**, lógica do Python e a memória do Ecossistema MyAI.

16.10 Função: **resultpython**

Esta função permite que um valor calculado ou manipulado dentro de um bloco de **Script (Python)** seja "devolvido" para o sistema e armazenado em uma variável de memória para uso em outras ações.

65. **Descrição:** Captura a saída de um script Python e a armazena em uma variável de memória.

66. **Parâmetros;**

bb. (ID) Identificador do agente

cc. Variável de memória que receberá o resultado do script

A Regra do **print("Ok :", ...)**

Para que o valor seja capturado com sucesso, você deve seguir esta sintaxe no seu código.

Por que isso é útil?

- **Cálculos Dinâmicos:** Somar valores de uma planilha para decidir se um boleto deve ser pago.
- **Tratamento de Texto:** Limpar espaços ou caracteres especiais de um nome capturado na web antes de salvar no banco de dados.
- **Validações Complexas:** Usar lógica Python para verificar se um CNPJ é válido e retornar "Sim" ou "Não" para o MyAI tomar uma decisão.

16.11. Eventos : Function :

readrepeaters16. Dicionário de Funções: Ação Function

A ação **Function** na **Ação readrepeaters**, esta ferramenta é o que permite a escalabilidade da sua automação, transformando um robô que processa um único dado em um sistema de processamento em massa (lote).

16.11 Função: readrepeaters

O módulo de **Repetidores** funciona como um banco de dados interno de entidades (clientes, empresas, CPFs) que o Agente deve processar de forma cíclica. Quando configurado, o Agente não executa a tarefa apenas uma vez, mas sim entra em um **loop automático**, repetindo todo o seu fluxo de trabalho para cada cliente cadastrado na lista selecionada.

67. **Descrição:** Lê uma informação específica do cadastro de repetidores e a escreve no processamento do Evento.

68. **Parâmetros;**

dd. (ID) Identificador do agente
ee. O número do repetidor.

- *Exemplo:* Usar **1** fará o Agente ler o **CNPJ / CPF** do cliente da vez.

Estrutura do Repetidor

Dentro da interface de Repetidores, você define uma **sequência de variáveis** para cada cliente. Por padrão, a configuração comum utiliza:

- **Sequência 1:** CNPJ / CPF.
- **Sequência 2:** Nome do Cliente.
- **Sequências Adicionais:** Podem ser configuradas conforme a necessidade (Inscrição Estadual, E-mail, Pasta de Destino, etc.).

16.12. Eventos : Function :

saverepeaters16. Dicionário de Funções: Ação Function

A ação **Function** na **Ação saverepeaters**, permite ao robô atualizar o cadastro de clientes em tempo real. Isso é fundamental para manter o histórico de processamento ou registrar informações colhidas durante a execução.

16.12 Função: **saverepeaters**

Enquanto a função anterior (**readrepeaters**) serve para o robô "ler" os dados do cliente, a **saverepeaters** serve para o robô "escrever" de volta no cadastro.

69. **Descrição:** Salva ou substitui uma informação específica dentro do cadastro de repetidores, vinculada ao cliente que está sendo processado no momento.

70. Parâmetros;

ff. (ID) Identificador do agente

gg. O número do repetidor.

hh. A informação que você deseja salvar. Pode ser um texto fixo ou uma variável.

- *Exemplo:* Usar 9 fará o Agente gravar o período do cliente da vez,
</40/></9/></122025/>

Exemplo de Uso: Controle de Processamento

Imagine que seu robô está baixando impostos. Você pode configurar o **saverepeaters** ao final do fluxo para marcar o sucesso.

16.13. Eventos : Function :

startdonwloads16. Dicionário de Funções: Ação Function

A ação **Function** na **Ação startdonwloads**, é o marco inicial de organização dentro de um fluxo de automação que envolve a captura de documentos. Ela resolve um problema comum: a "bagunça" na pasta de Downloads do Windows, garantindo que o robô saiba exatamente quais arquivos pertencem a qual cliente e a qual período.

16.13 Função: startdonwloads

Esta função prepara o ambiente para o recebimento de novos arquivos. Ela "limpa" a visão do robô sobre a pasta de downloads, permitindo que, após o download ser concluído, o sistema consiga isolar apenas os novos arquivos baixados para processá-los.

71. **Descrição:** Monitora o início de uma sessão de download. O app realiza um filtro inteligente para separar os arquivos baixados, permitindo que eles sejam classificados corretamente por **Repetidor (Cliente)** e **Período**.

72. **Parâmetros;**

Por que usar o startdownloads?

Sem essa função, se a pasta de downloads já contiver 100 arquivos, o robô teria dificuldade em saber qual deles é o novo. O **startdownloads** faz com que o Agente foque apenas no **conteúdo da execução atual**, evitando:

- Envio de documentos errados para clientes.
- Duplicidade de arquivos.
- Erros de processamento por arquivos "restantes" de automações anteriores.

16.14. Eventos : Function :

savepathdonwloadsrep..16. Dicionário de Funções: Ação Function

A ação **Function** na **Ação savepathdonwloadsrepeaters**, é o fechamento lógico e operacional do processo de captura. Se o **startdownloads** abriu o monitoramento, esta função é quem executa a "inteligência de arquivo", movendo, organizando e marcando o trabalho como concluído no cadastro de repetidores, podendo ser usado armazenamento em nuvem (Google Drive).

16.14 Função: **savepathdonwloadsrepeaters**

Esta função finaliza a sessão de download, retira os arquivos da pasta temporária e os organiza na estrutura de pastas do cliente, além de realizar o controle de competência (período).

73. **Descrição:** Finaliza o monitoramento de downloads, move os arquivos para o caminho específico do cliente e atualiza o status de período para evitar duplicidade.

74. **Parâmetros;**

- ii. (ID) Identificador do agente
- jj. Indica qual sequência no cadastro de repetidores contém o caminho base do cliente.

Exemplo: Se na Sequência 5 estiver **C:\Documentos\empresa_luiz**, o robô usará este destino.

- kk. Define uma subpasta para classificação do tipo de documento.

Exemplo: **</Federais/>**. O robô criará essa pasta dentro do caminho do cliente.

- ll. Indica a sequência (ex: 9) que controla o período. Isso permite que o app saiba que aquele mês já foi baixado para aquele cliente.

Estrutura Final do Arquivo

Ao final da execução desta função, sua estrutura de arquivos no Windows ficará perfeitamente organizada sem intervenção manual:

Caminho Final: **C:\Documentos\empresa_luiz\Federais\122025\abc.pdf**

16.15. Eventos : Function :

deletopathdonwloadsr..16. Dicionário de Funções: Ação Function

A ação **Function** na **Ação deletopathdonwloadsrepeaters**, atua como o "faxineiro" do processo de captura. Sua principal missão é garantir que a área de trabalho do robô esteja limpa, evitando que arquivos antigos ou resíduos de tentativas falhas interfiram na execução atual.

16.15 Função: **deletopathdonwloadsrepeaters**

Diferente das funções de salvamento, esta é uma ação de **limpeza preventiva**. Ela é frequentemente utilizada no **início** do fluxo de repetição ou logo após uma falha detectada, para apagar resíduos que sobraram na pasta temporária.

75. Descrição: Localiza e remove arquivos ou pastas residuais dentro do diretório de downloads do Agente, preparando o terreno para uma nova captura limpa.

76. Parâmetros;

mm. (ID) Identificador do agente

nn. Indica qual sequência no cadastro de repetidores contém o caminho base do cliente.

Exemplo: Se na Sequência 5 estiver **C:\Documentos\empresa_luiz**, o robô usará este destino.

oo. Define uma subpasta para classificação do tipo de documento.

Exemplo: **</Federais/>**. O robô criará essa pasta dentro do caminho do cliente.

pp. Indica a sequência (ex: 9) que controla o período. Isso permite que o app saiba que aquele mês já foi baixado para aquele cliente.

Por que apagar antes do fechamento?

Existem dois cenários críticos onde o uso desta função é essencial:

- 1. Limpeza de Resíduos:** Se um download anterior foi interrompido (queda de internet ou erro no site), o Windows pode manter arquivos temporários (como **.crdownload**). Esta função remove esses "lixos" para que o **startdownloads** seguinte monitore apenas arquivos reais e novos.
- 2. Reset de Tentativa:** Se o robô precisa reprocessar um cliente que deu erro no meio do caminho, o **deletopathdonwloadsrepeaters** limpa a pasta para garantir que não haja confusão entre o que foi baixado na primeira tentativa e o que será baixado agora.

16.16. Eventos : Function :

emailpathdonwloadsr..16. Dicionário de Funções: Ação Function

A ação **Function** na **Ação emailpathdonwloadsrepeaters**, é o braço de comunicação do seu Agente, permitindo que, após baixar e organizar os arquivos, o robô os envie diretamente para os e-mails cadastrados de cada cliente de forma personalizada, identifica o período em processamento atual.

16.16 Função: emailpathdonwloadsrepeaters

Esta função automatiza o envio de e-mails utilizando os arquivos contidos em uma pasta específica (geralmente a que foi feita pela função) **savepathdonwloadsrepeaters**.

77. Descrição: Envia um e-mail com o conteúdo de uma pasta de downloads específica, utilizando os dados de contato vinculados ao repetidor (cliente) em processamento.

78. Parâmetros;

qq. (ID) Identificador do agente

rr. Sequência que contém o caminho base (ex: **5** para a pasta do CNPJ).

Exemplo: Se na Sequência 5 estiver **C:\Documentos\empresa_luiz**, o robô usará este destino.

ss. Define uma subpasta para classificação do tipo de documento.

Exemplo: **</Federais/>**. O robô criará essa pasta dentro do caminho do cliente.

tt. Sequência com o nome da empresa para personalização (ex: **2**).

uu. Sequência com email 1 do cliente no cadastro (ex: **6**).

vv. Sequência com email 2 do cliente no cadastro (ex: **7**).

ww. O assunto do email que o cliente visualizará.

xx. Filtro de arquivos a serem anexados (ex: ***.*** para todos os arquivos da pasta).

O Fluxo de Comunicação Completo

Com esta função, o seu ecossistema atinge a autonomia total:

1. **Limpeza:** **deletopathdonwloadsrepeaters** limpa resíduos.
2. **Start:** **startdonwloads** dá partida
3. **Identificação:** **readrepeaters** lê o cliente e seus e-mails.
4. **Captura:** O Agente navega e baixa os impostos.
5. **Organização:** **savepathdonwloadsrepeaters** guarda na pasta "Federal".
6. **Entrega:** **emailpathdonwloadsrepeaters** anexa tudo da pasta "Federal" e envia para os e-mails das sequências 6 e 7.

16.17. Eventos : Function :

updateperiodrepeaters

16. Dicionário de Funções: Ação Function

A ação **Function** na **Ação updateperiodrepeaters**, faz a gravação do período processado.

16.17 Função: **updateperiodrepeaters**

Esta função é o "carimbo final" de uma tarefa bem-sucedida. Ela é utilizada para atualizar especificamente a informação de período no cadastro de repetidores, garantindo que o controle de competência mencionado anteriormente seja atualizado em tempo real.

79. **Descrição:** Salva o período de processamento atual diretamente na sequência correspondente dentro do cadastro de repetições.

80. **Parâmetros;**

yy. (ID) Identificador do agente

zz. Sequência cadastro em repetidos que tem o período (ex: 9).

Por que usar **updateperiodrepeaters** em vez de **saverepeaters** comum?

Embora a função **saverepeaters** possa escrever qualquer texto, a **updateperiodrepeaters** é otimizada para o **controle de fluxo de tempo**. Ela trabalha em conjunto com as funções de download para garantir que o "Período" seja uma variável de controle lógico e não apenas um texto estático.

OBS. Esta informação é arquivada em agentes (período).

16.18. Eventos : Function :

goservices16. Dicionário de Funções: Ação Function

A ação **Function** na **Ação goservices**, ferramenta de "salto" ou "redirecionamento" que permite criar automações não lineares e inteligentes.

16.18 Função: goservices

Esta função permite que o Agente interrompa a sequência linear atual e salte para qualquer outro ponto do microserviço (ou volte ao início dele). É essencial para criar loops personalizados, tratar erros ou repetir validações sem precisar reiniciar todo o Agente.

81. **Descrição:** Faz o fluxo de execução saltar para o número de sequência de um microserviço específico.

82. **Parâmetros;**

aaa. (ID) Identificador do microserviço

Aplicação Prática: O "Loop de Validação"

Imagine que seu robô está tentando baixar um certificado e o site apresenta uma instabilidade temporária:

1. O robô tenta o download.
2. Uma função **locatereplaceHTML** verifica que a página de erro apareceu.
3. O Agente então usa o **goservices** com o parâmetro **</1/>** (ou o número da linha de login).
4. **Resultado:** O robô volta ao começo do microserviço para tentar novamente, em vez de simplesmente falhar e parar.

Dica de Especialista

Use o **goservices** em conjunto com uma variável de "contador" (setada via **savevariable**). Assim, você pode dizer ao robô: *"Tente voltar ao começo e repetir este passo no máximo 3 vezes; se na quarta vez ainda der erro, siga em frente ou pare"*. Isso traz uma robustez incrível para lidar com sites lentos ou instáveis.

16.19. Eventos : Function :

certificado

16. Dicionário de Funções: Ação

Function

A ação **Function** na **Ação certificado**, resolve um dos maiores "gargalos" da automação web: a interação com janelas nativas do Windows para seleção de **Certificados Digitais (A1/A3)**.

16.19 Função: **certificado**

Esta função automatiza a escolha do certificado correto dentro da lista de certificados instalados no repositório do Windows, eliminando a necessidade de intervenção humana quando o site solicita a autenticação digital.

83. Descrição: Localiza e seleciona o certificado digital correspondente ao cliente (repetidor) em processamento, realiza a busca na lista de repositório de certificados do windows, organiza por nome e confirma a seleção.

84. Parâmetros;

bbb. (ID) Identificador do agente
ccc. Sequência que contém o CNPJ / CPF (ex: 1) em repetidores.

A Inteligência por trás da Função

Para que essa "mágica" aconteça, o MyAI realiza um fluxo complexo em segundos:

1. **Sincronização com Repetidores:** O robô lê o nome do cliente que está no cadastro de **Repetidores**.
2. **Mapeamento:** Ele cruza esse nome com o **Cadastro de Certificados** mapeado no sistema.
3. **Ordenação:** Ao abrir a janela de certificados no Google Chrome, o robô organiza a lista alfabeticamente para garantir a previsibilidade.
4. **Localização:** Ele executa uma busca interna na lista para encontrar o certificado cujo nome seja igual ao do repetidor.
5. **Acesso:** Uma vez localizado, ele seleciona o certificado correto e simula o **Enter**, efetuando o login no portal (como e-CAC, Prefeituras ou Juntas Comerciais).

16.20. Eventos : Function :

readdaterepeaters16. Dicionário de Funções: Ação Function

A ação **Function** na **Ação readdaterepeaters**, é essencial para fornecer ao módulo de **Repetidores** uma camada de controle temporal que, nativamente, ele não possui, permitindo o registro preciso de quando uma tarefa iniciou e quando foi concluída para cada cliente.

16.20 Função: readdaterepeaters

Esta função atua como um marcador de tempo dinâmico, traduzindo o momento da execução em uma data formatada que é gravada diretamente no cadastro de repetidores.

85. Descrição: Escreve a variável de repetição em forma de período (inicial ou final), permitindo que o cadastro de repetidores tenha o controle do histórico de execução que antes era restrito apenas ao log do Agente.

86. Parâmetros;

- ddd. (ID) Identificador do agente
- eee. Sequência que contém o período (ex: **9**) em repetidores.
- fff. Define se o robô está marcando o começo ou o fim da tarefa.

</I/> : Grava a **Data Inicial** (usado no posicionamento para iniciar a tarefa).

</F/> : Grava a **Data Final** (usado no encerramento após a conclusão).

Lógica de Controle de Execução

Como o Agente possui controle interno, mas os Repetidores não, essa função preenche essa lacuna de gestão:

1. **Início do Processo:** Logo após o **readrepeaters** identificar o cliente, você utiliza o **readdaterepeaters** com o parâmetro **</I/>**. Isso registra no painel que o robô "começou a trabalhar" naquela empresa específica.
2. **Durante a Tarefa:** O robô executa todos os downloads e navegações.
3. **Conclusão:** Após o envio do e-mail ou salvamento do arquivo, utiliza-se o parâmetro **</F/>** para gravar a data de conclusão.

Benefício Estratégico: Isso permite que o gestor filtre no cadastro de repetidores quais empresas foram concluídas no dia de hoje e quanto tempo, em média, o robô levou para processar cada uma, transformando o cadastro em um verdadeiro **dashboard de produtividade**.

16.21. Eventos : Function :

basicmathematical16. Dicionário de Funções: Ação Function

A ação **Function** na **Ação basicmathematical**, manipulação de dados e variáveis de controle, permite ao robô realizar operações aritméticas essenciais durante a execução.

16.21 Função: **basicmathematical**

Esta função é o motor de cálculo do Agente. Ela é fundamental para criar contadores de páginas, somar valores extraídos de tabelas ou realizar divisões para cálculos de médias e impostos.

87. **Descrição:** Realiza cálculos matemáticos entre valores fixos ou variáveis de memória, salvando o resultado em uma variável de destino.

88. **Parâmetros;**

- ggg. (ID) Identificador do agente
- hhh. Define o tipo de cálculo: (**A**: Adição, **S**: Subtração, **D**: Divisão, **M** Multiplicação)
- iii. Valor atual ou nome da variável de memória inicial.
- jjj. Valor a ser somado, subtraído, etc., ou outra variável de memória.
- kkk. A variável de memória que receberá o resultado final.

Aplicação Prática: Criando um Contador (Incremento)

O uso mais comum para esta função é o **incremento de páginas** ou de itens processados (o famoso **++** da programação).

Exemplo: `</29/></A/></sysLoopPage/></1/></sysLoopPage/>`.

1. O robô lê o valor atual da variável `sysLoopPage`(ex: 1).
2. Realiza a **Adição (A)** com o valor fixo **1**.
3. Grava o novo valor (**2**) de volta na variável `sysLoopPage`

Por que usar **basicmathematical**?

- **Navegação Dinâmica:** Útil para avançar páginas em sites onde o link é numérico (ex: `pagina=1`, `pagina=2`).
- **Validação de Dados:** Permite somar valores de boletos e conferir se o total bate com o esperado antes de prosseguir.
- **Controle de Tentativas:** Usado com o `goservices` para contar quantas vezes o robô tentou executar um passo e decidir se deve parar ou continuar.

16.22. Eventos : Function :

roundmathematical16. Dicionário de Funções:

Ação Function

A ação **Function** na **Ação roundmathematical**, é o complemento indispensável da **basicmathematical**, garantindo que os resultados financeiros ou estatísticos possuam a precisão necessária para o armazenamento em bancos de dados ou preenchimento de formulários.

16.22 Função: roundmathematical

Esta função é utilizada para arredondar valores numéricos contidos em variáveis de memória, limitando a quantidade de casas decimais após a vírgula.

89. **Descrição:** Arredonda um cálculo ou valor específico para o número de casas decimais desejado.

90. **Parâmetros;**

- III. (ID) Identificador do agente
- mmm. Valor atual ou nome da variável de memória inicial.
- nnn. A quantidade de dígitos após a vírgula.
- ooo. A variável de memória que receberá o resultado final.

Exemplo de Uso Integrado

Imagine que o robô realizou uma divisão de impostos e o resultado foi 150.33333333

1. **basicmathematical:** Realiza a conta e salva em **sysImposto**
2. **roundmathematical:** Recebe **sysImposto**, define **2** casas decimais e salva o valor final **150.33** na mesma variável.

16.23. Eventos : Function : endloop16.

Dicionário de Funções: Ação Function

A ação **Function** na **Ação endloop**, é a peça que faltava para criar condições de parada ou repetição baseadas em variáveis dinâmicas.

16.23 Função: endloop

Diferente do **goservices** que realiza um salto direto, o **endloop** é uma função de controle condicional. Ela decide se o robô deve continuar avançando ou se deve retornar ao topo do microserviço atual com base na comparação de dois valores.

91. **Descrição:** Verifica se uma variável é menor ou igual a outra variável. Se a condição for verdadeira, o Agente volta automaticamente ao topo do microserviço atual.

92. **Parâmetros;**

ppp. (ID) Identificador do agente
qqq. A variável de controle.
rrr. A variável de limite para a repetição

O Ciclo de Vida de um Loop Manual

Ao combinar as funções que aprendemos, você cria um fluxo indestrutível:

1. **Início:** O robô entra no microserviço.
2. **Cálculo:** A função **basicmathematical** incrementa o contador (ex: **sysLoopPage = sysLoopPage + 1**).
3. **Ação:** O robô executa a tarefa (clique, download, etc.).
4. **Decisão (endloop):** O robô compara: "O **sysLoopPage** ainda é menor ou igual ao limite?".
 - **Sim:** Ele volta ao topo do microserviço e repete.
 - **Não:** Ele encerra o loop e segue para a próxima etapa da automação.

16.24. Eventos : Function :

downloadurl16. Dicionário de Funções: Ação Function

Permite que o robô baixe arquivos diretamente através de uma URL, sem a necessidade de simular cliques em botões de download complexos ou lidar com caixas de diálogo do navegador.

16.24 Função: **downloadurl**

A função **downloadurl** captura o link de um arquivo e realiza o download de forma direta para a pasta de destino configurada.

93. **Descrição:** Faz o download de um arquivo a partir de uma URL específica.

94. **Parâmetros;**

- a. O link direto para o arquivo
- b. A pasta onde o arquivo será gravado.
- c. O identificador do do agente.
- d. Variável de memória que armazenará o resultado.
- e. O conteúdo gravado se o texto for encontrado
- f. O conteúdo gravado se o texto NÃO for encontrado

16.25. Eventos : Function :

updateperiodagents16. Dicionário de Funções:

Ação Function

A ação **Function** na **Ação updateperiodagents**, faz a gravação do período processado.

16.25 Função: **updateperiodagents**

Esta função é o "carimbo final" de uma tarefa bem-sucedida. Ela é utilizada para atualizar especificamente a informação de período no cadastro de agentes e **tem prioridade sobre cadastro de repetidores**, garantindo que o controle de competência mencionado anteriormente seja atualizado em tempo real.

95. **Descrição:** Salva o período de processamento atual diretamente na sequência correspondente dentro do cadastro de agentes.

96. **Parâmetros;**

sss. (ID) Identificador do agente

Obs. O **updateperiodagents** deve ser usado antes **updateperiodrepeaters**

16.26. Eventos : Function : endpoint16.

Dicionário de Funções: Ação Function

A ação **Function** na **Ação endpoint**, permite ao MyAI conversar com outros sistemas, APIs de bancos, gateways de SMS ou serviços em nuvem, enviando dados e recebendo respostas em tempo real.

16.26 Função: endpoint

Esta função realiza requisições HTTP (geralmente do tipo POST) para enviar informações processadas pelo robô para uma URL externa, utilizando o formato padronizado JSON.

Parâmetros do Comando

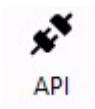
A sintaxe segue o padrão de tags separadas por delimitadores:

Exemplo = `</88/></1/></http://<<ipapi>>:<<portapi>>/ /></"to": "81988952851",
"message": "Teste SMS MyAI"/></sysResposta/>`

1. **ID do Agente:** Identificação do robô que está disparando a ação.
2. **ID da API:** Referência ao cadastro técnico da API (onde ficam guardados o IP, Porta e Token).
3. **URL do Endpoint:** O endereço destino (ex: `http://api.gateway.com/send`).
4. **Corpo (Mensagem):** O conteúdo que será convertido em JSON. Ex: `{"to": "5511...", "message": "Texto"}`.
5. **Variável de Memória:** O nome da variável onde a resposta do servidor (ex: "Sucesso" ou "Erro 404") será salva para uso posterior (ex: `</sysResposta/>`).

Cadastro e Máscaras de API

Para facilitar a manutenção, o Ecossistema utiliza um sistema de variáveis globais no cadastro de APIs. Isso significa que, se você mudar o IP do seu servidor, não precisa editar todos os microsserviços, apenas o cadastro central.



17.1. Eventos : Browser :

startbrowser17. Dicionário de Funções: Ação Browser

Módulo de navegação web, vamos detalhar o comando fundamental para qualquer automação baseada em navegador: o **startbrowser**. Esta função é o ponto de partida para a interação do MyAI com portais e sites, web scraping de sites dinâmicos.

17.1 Função: **startbrowser**

Esta função é responsável por instanciar e abrir o navegador Google Chrome utilizando a tecnologia Selenium, preparando o ambiente para a execução dos scripts de automação.

97. **Descrição:** Abre o navegador Chrome em uma URL específica, permitindo configurações avançadas de visibilidade e evasão de bloqueios.

98. **Parâmetros;**

- a. O endereço do site que o robô deve acessar (ex: `</ https://www.google.com.br/>`).
- b. Serviço anti-bot **não remove a mensagem O Chrome está sendo controlado por um software de teste automatizado**, mas o script pode ser copiado para IDLE ou fazer um .py e executado, você pode importar outro pacote (**undetected-chromedriver**) para conseguir remover essa mensagem.
- c. **Maximizar:** Abre o navegador em tela cheia para facilitar a localização de elementos visuais.
- d. **Minimizar:** Mantém a janela oculta na barra de tarefas.
- e. **Headless (Segundo Plano):** Executa a navegação sem abrir uma janela visível, ideal para servidores ou tarefas de background.

Regra de Ouro: O Encerramento do Ciclo

Toda vez que uma automação inicia um navegador com o **startbrowser**, ela **obrigatoriamente** deve terminar com a função **quitbrowser**.

- **Por que é vital?** Sem o **quitbrowser**, as instâncias do Chrome e do ChromeDriver permanecem abertas na memória RAM do computador. Após várias repetições de clientes, isso pode causar lentidão extrema ou o travamento total do sistema por falta de recursos.

Inteligência Anti-Bot e Código Embarcado

Um diferencial crucial do MyAI. Como muitos sites modernos possuem proteções contra automação (anti-bots), o MyAI utiliza scripts Python customizados para modificar as propriedades do navegador (como o **AutomationControlled**), fazendo com que o Selenium "não pareça" um robô para o site, mas

17.2. Eventos : Browser :

existelement17. Dicionário de Funções: Ação Browser

Esta função permite que o Agente tome decisões baseadas na presença real de componentes na página, motor `driver.find_element`.

17.2 Função: **existelement**

A função **existelement** é utilizada para pesquisar a existência de um elemento HTML específico no navegador e gravar um resultado customizado em uma variável, dependendo se o objeto foi encontrado ou não.

99. **Descrição:** Pesquisa um elemento e salva o resultado em uma variável de memória.

100. **Parâmetros;**

- a. O identificador do objeto instanciado.
- b. O método de localização do elemento;
 - `CLASS_NAME`, `CSS_SELECTOR`, `ID`, `NAME`, `LINK_TEXT`, `TAG_NAME` ou `XPAT`
- c. O seletor ou nome do elemento a ser buscado
- d. ID numérico do Agente em execução ou frase = MSG abre ação de mensagem (caso entre o elemento ex. se existir Captcha abre mensagem)
- e. Nome da variável de memória que receberá o status ou Título
- f. O conteúdo que será gravado se o elemento **existir** ou Texto da mensagem
- g. O conteúdo que será gravado se o elemento **não existir**

Aviso de Erro Fatal: Esta função é crítica. Caso o elemento configurado não seja encontrado e não haja um tratamento preventivo, o Agente gerará um erro fatal, paralisando a execução imediatamente para evitar ações em páginas incorretas (**se o ID do agente não for preenchido**).

Aplicação Estratégica

Você pode usar o **existelement** em conjunto com o **goservices** para criar esperas inteligentes. Por exemplo: "Verifique se o botão 'Download' apareceu. Se **true**, clique; se **false**, use o **goservices** para voltar ao topo e tentar carregar a página novamente".

17.3. Eventos : Browser :

findelement17. Dicionário de Funções: Ação Browser

Representa as "mãos" e a "inteligência operacional". É através dela que o MyAI efetivamente interage com os elementos da página para executar tarefas, motor `find_element + clear + click + send_keys + get_attribute`

17.3 Função: **findelement**

A função **findelement** localiza um elemento HTML e cria um objeto em memória que pode ser manipulado através de **Eventos** e **Comandos**.

101. **Descrição:** Pesquisa um elemento e cria um objeto para ser manipulado
102. **Parâmetros (Aba Navegação);**
 - a. O identificador do objeto instanciado.
 - b. O método de localização do elemento;
 - `CLASS_NAME`, `CSS_SELECTOR`, `ID`, `NAME`, `LINK_TEXT`, `TAG_NAME` ou `XPAT`
 - c. O seletor ou nome do elemento a ser buscado

Captura Inteligente (Tecla "M")

Para facilitar o mapeamento, na aba **Navegação**, você pode posicionar o mouse sobre o elemento desejado no navegador e teclar "M". O MyAI detectará automaticamente o elemento e preencherá o campo.

Eventos: Ação sobre o Elemento

Após localizar o objeto, você define o que o robô deve fazer com ele:

- **Click:** Executa um clique no botão ou link.
- **Clear:** Apaga o conteúdo existente em um campo de texto.
- **Send Keys:** Escreve um texto ou aciona teclas especiais (como `Enter`, `Tab`, `Esc`).

17.3.1. Eventos : Browser :

17.3.1 Comandos: Refinamento do Resultado

O campo de Retorno permite que o conteúdo obtido do elemento passe por uma série de comandos para melhorar o resultado final (Modelagem de Dados):

17.3.1.1 Comando: **searchtext**

O **searchtext** permite que você localize padrões dentro do texto capturado e aplique regras de extração ou substituição imediata.

Estrutura : **searchtext**(</Pesquisa/></Substituto/><///></Tipo/>)

Exemplo Retorno = Olá mundo 12,000.11

Tabela de Operações (**Tipo**)

P = **Posição** Retorna a posição numérica onde o texto começa.

searchtext(</Olá/><///><///></P/>) = Resultado 1

TL = **Texto à Esquerda** Pega tudo o que vem antes do texto

searchtext(</mundo/><///><///></TL/>) = Resultado Olá

TR = **Texto à Direita** Pega tudo o que vem após o texto.

searchtext(</Olá/><///><///></TR/>) = Resultado mundo 123

N = **Somente Números** Filtra apenas os dígitos no o texto.

searchtext(</mundo/><///><///></N/>)= Resultado 123

M= **Monetário** Formata valores financeiros no o texto.

searchtext(</Olá/><///><///></P/>) = Resultado 1

S = **Substituir** Troca o termo buscado pelo texto substituto.

searchtext(</mundo/></MyAI/><///></S/>)= Resultado Olá MyAI 123

17.3.2. Eventos : Browser :

findelement17.3.2.1 Comando: **variablereplace**

o **variablereplace** é um comando de **atribuição dinâmica**. Ele permite ler o estado atual do dado processado e salvá-lo em uma variável específica.

Estrutura : **variablereplace**

Parâmetros;

- a. (ID) Identificador do agente
- b. A variável de memória que receberá o resultado final.
- c. Resultado processado;

Se deixado **em branco** (<///>): O comando grava automaticamente o resultado atual das transformações até aquele momento.

Se **preenchido**: O comando ignora o pipeline anterior e grava exatamente a informação digitada neste parâmetro.

Exemplo de Uso Prático (Encadeamento)

Imagine que você está capturando o valor de uma nota fiscal. No campo **Retorno**, você pode encadear os comandos assim:

1. **searchtext(</Valor: /><///><///></M/>)** → Isola o número e formata como monetário (ex: 150,00).
2. **variablereplace(</42/></sysValorNF/><///>)** → Pega esse 150,00 e o salva na variável **sysValorNF**.

17.3.3. Eventos : Browser : findelement

17.3.3.1 Comando: **selectlist**

Esta função é especializada em interagir com campos de seleção (combobox/dropdown), que são muito comuns em portais governamentais para escolha de estados, meses ou tipos de impostos, permitindo escolher uma opção específica dentro de uma lista suspensa de forma direta.

Estrutura : **selectlist**

Parâmetros;

- d. Seleciona um item específico dentro de um elemento HTML de seleção.
- e. O tipo de identificador do campo select (ID, XPATH, NAME, etc.).
- f. O conteúdo do identificador

17.3.4. Eventos : Browser : findelement

17.3.4.1 Comando: **getattribute**

Esta função ler a informação da propriedade do elemento.

Estrutura : **getattribute**

Parâmetros;

- g. Nome da propriedade que você deseja capturar.

Exemplos comuns:

href: Para pegar o link de um botão ou imagem.

value: Para pegar o que está escrito dentro de um campo de formulário.

src: Para pegar o caminho de uma imagem ou frame.

title: Para pegar o texto que aparece ao passar o mouse.

- h. O tipo de identificador do campo select (ID, XPATH, NAME, etc.).
- i. O conteúdo do identificador

Exemplo Real: Um site tem um botão de "Download" que, ao ser clicado, abre uma página complexa. Com **getattribute**, você pode capturar o atributo **href** desse botão.

1. O robô extrai o link direto (ex: **https://site.com/arquivo.zip**).
2. Esse link vai para uma variável de memória.
3. Você usa a função **downloadurl** para baixar o arquivo instantaneamente, sem precisar clicar em nada.

17.3.5. Eventos : Browser : findelement

17.3.5.1 Comando: **script**

Esta função ler um código em python

Estrutura : **script**

código python puro

17.3.6. Eventos : Browser : findelement

17.3.6.1 Comando: **readervariablescr**

Grava o conteúdo de uma variável de memória para uma variável python

Estrutura : **readervariablescr**

Parâmetros;

- j. Id do agente.
- k. Nome de variável de memória no MyAI
- l. Nome da variável a ser criada em python com o conteúdo no MyAI

17.3.7. Eventos : Browser :

findelement17.3.7.1 Comando: `tabledataframe`

Ler uma tabela em HTML e transforma em um dataframe para posterior exportação para banco de dados, csv ou excel com uso das funções `databaseexitconnection` e `exportdataframe`

Estrutura : `tabledataframe`

Parâmetros;

- m. Nome do dataframe a ser criado a partir da tabela HTML
- n. Sequência de tabelas que existe na página. ex. 0 indica a primeira tabela a ser convertida

17.3.8. Eventos : Browser : findelement

17.3.8.1 Comando: **scrollpage**

Rola a página html pois existem páginas dinâmicas que só aparece o elemento após a rolagem da tela

Estrutura : **scrollpage**

Parâmetros;

- o. Descrição a procurar na rolagem da tela

17.4. Eventos : Browser : findtext17.

Dicionário de Funções: Ação Browser

A função **findtext** realiza uma varredura textual em toda a estrutura visível da página., `motor driver.page_source`.

17.4 Função: **findtext**

Esta função é ideal para validar mensagens de sistema, alertas de erro ou confirmações de sucesso que aparecem flutuando na página **sem um ID fixo**.

- 103. **Descrição:** Pesquisa um texto dentro do HTML
- 104. **Parâmetros;**
 - a. O identificador do do agente.
 - b. Texto que o robô deve procurar na página
 - c. Variável de memória que armazenará o resultado.
 - d. O conteúdo gravado se o texto for encontrado
 - e. O conteúdo gravado se o texto NÃO for encontrado

Aplicação Prática: Validação de Processamento

Esta função é extremamente útil para fluxos de decisão após um clique:

1. O robô clica em "Enviar".
2. A função **findtext** procura pela frase "Operação realizada com sucesso".
3. Se encontrar, grava **OK** na variável `sysStatus`.
4. O Agente então usa essa variável para decidir se deve seguir para o próximo `repetidor` ou se deve tentar novamente usando o `goservices`

17.5. Eventos : Browser :

quitbrowser17. Dicionário de Funções: Ação Browser

Para garantir que o ambiente de trabalho fique limpo e que os recursos do computador não sejam esgotados por processos "zumbis", vamos detalhar a função de encerramento do navegador, incluindo sua estratégia de contingência.

17.5 Função: **quitbrowser**

Esta função é o comando padrão para encerrar a instância do navegador que foi aberta pelo Selenium, **caso use serviço anti-bot pode ser que não funcione corretamente o fechamento e tenha que fechar via tecla ALT + F4.**

105. **Descrição:** Fecha o navegador Google Chrome e encerra o processo do driver de comunicação.

Contingência: O "Fechamento Forçado"

Em alguns portais, scripts internos do site ou janelas de alerta (pop-ups) podem impedir que o comando padrão de fechar funcione corretamente. Para esses casos, o MyAI possui uma estratégia de segurança baseada em comandos de teclado nativos:

1. **Foco no Título:** O robô clica no título da janela para garantir que ela seja a aplicação ativa no Windows.
2. **Comando de Sistema:** O Agente simula a combinação de teclas **ALT + F4**.
3. **Resultado:** Isso força o encerramento da janela pelo Sistema Operacional, ignorando travamentos do Selenium ou scripts de "deseja realmente sair?" do site.

Por que isso é importante no loop de Repetidores?

Se o seu Agente processa 100 empresas (repetidores) e falha ao fechar o navegador em 10 delas, você terminará o dia com 10 janelas do Chrome abertas. Isso causará:

- **Consumo de RAM:** O computador ficará lento e poderá travar o Agente.
- **Conflito de Cache:** O próximo cliente pode tentar usar a sessão do cliente anterior.

17.6. Eventos : Browser :

maximizebrowser17. Dicionário de Funções: Ação Browser

A função **maximizebrowser** modifica a interface do Ecosystema.

17.6 Função: **maximizebrowser**

Esta função envia um comando direto ao driver do navegador para expandir a janela para o tamanho máximo da tela disponível.

Quando utilizar o **maximizebrowser**?

Embora pareça uma ação simples, ela é estratégica em três cenários:

1. **Garantia de Layout Desktop:** Alguns sites ocultam barras de ferramentas ou menus laterais em janelas reduzidas. Maximizar garante que o **findelement** encontre os seletores (ID, XPATH) no local padrão.
2. **Capturas de Tela (Screenshots):** Se você utiliza funções de auditoria visual, uma janela maximizada captura mais informações da página em um único arquivo.
3. **Resolução de Elementos Sobrepostos** Em janelas pequenas, elementos como "avisos de cookies" ou "chats" podem cobrir botões de ação. O redimensionamento para tela cheia ajuda o robô a contornar esses obstáculos.

17.7. Eventos : Browser :

maximizebrowser17. Dicionário de Funções:

Ação Browser

A função **minimizebrowser** modifica a interface do Ecossistema.

17.7 Função: **minimizebrowser**

Esta função envia um comando direto ao driver do navegador para minimizar a janela.

Quando utilizar o **minimizebrowser**?

1. Proteção contra Intervenção Humana

O robô usa mouse e teclado, ele precisa de exclusividade. Se por algum motivo alguém precisar usar o computador rapidamente enquanto o robô está em execução, o **minimizebrowser** "tira da frente" a janela do Chrome, diminuindo o risco de o usuário clicar acidentalmente em algo dentro do site da automação.

2. Consumo de Recursos de Vídeo

Em servidores ou máquinas com pouco processamento gráfico, manter o navegador minimizado pode reduzir levemente o esforço de renderização da interface, focando o processamento apenas na lógica do processamento.

17.8. Eventos : Browser :

switchwindow17. Dicionário de Funções: Ação Browser

A função **switchwindow** atua como a "ponte" necessária para que o Agente não se perca quando o site abre novos conteúdos.

17.8 Função: switchwindow

Sempre que uma ação (como clicar em um link de "Ajuda", "Imprimir" ou "Detalhes") abre uma nova aba, o Selenium continua "preso" na aba anterior até que você ordene a troca.

- **Índice 0 (Janela Pai):** É a sua base de operações. Onde o robô começou a navegação.
- **Índice 1 (Janela Filho):** É a nova aba que surgiu. Sem mudar para o índice 1, qualquer comando de **findelement** ou **clickbrowser** falhará, pois o robô estará procurando o elemento na página errada.

Exemplo de Script Estável

1. **Ação** **clickbrowser** no link que abre o relatório em PDF.
2. **Ação** **switchwindow** com índice **</1/>** para assumir o controle da nova aba.
3. **Ação** **downloadurl** para capturar o documento.
4. **Ação** **closewindow** para fechar a aba do relatório.
5. **Ação** **switchwindow** com índice **</0/>** para voltar à página principal e continuar o loop.

17.9. Eventos : Browser :

closewindow17. Dicionário de Funções: Ação Browser

A função **closewindow**. Ela é a ferramenta de "limpeza seletiva" do seu robô durante a navegação.

17.9 Função: **closewindow**

Esta função fecha exclusivamente a janela ou aba que detém o foco do Selenium no momento da execução.

Regras Críticas de Fluxo

Diferente de outras funções, a **closewindow** exige atenção redobrada à sequência de comandos:

1. **O "Vácuo" de Foco:** Assim que você executa o **closewindow** o robô fica em um estado "cego", sem saber para qual janela olhar, mesmo que a janela principal (Pai) ainda esteja aberta.
2. **Ação Obrigatória: Sempre** utilize a função **switchwindow** imediatamente após um **closewindow** para devolver o foco à janela principal (Índice 0) ou a outra aba que deva continuar o processo.

17.10. Eventos : Browser :

openwindow17. Dicionário de Funções: Ação Browser

Esta função permite que o robô tome a iniciativa de expandir sua navegação para novos endereços sem perder o progresso da janela atual.

17.10 Função: **openwindow**

Diferente de clicar em um link que abre uma nova aba automaticamente, o **openwindow** é um comando direto para o navegador criar um novo espaço de navegação para uma URL específica

Descrição: Abre uma nova janela (ou aba) no navegador Google Chrome que já está sendo controlado pelo Agente.

Parâmetros: O link da URL completo que deve ser aberto (ex: `</https://www.google.com/>`). Você também pode usar as **Tags Dinâmicas** (como `{{ReaderVariable[...]}}`) para abrir links capturados anteriormente.

Por que usar o **openwindow**?

Esta função é extremamente útil em fluxos de trabalho que envolvem comparação ou transferência de dados entre dois sites diferentes:

1. **Multitarefa:** Você pode manter o Portal do Governo aberto na Janela 0 e abrir o site da sua empresa na Janela 1 para conferir se os dados batem.
2. **Navegação Paralela:** Se um site é muito lento para carregar, o robô pode abrir uma nova janela para realizar outra tarefa rápida enquanto aguarda o carregamento da primeira.
3. **Links Dinâmicos:** Quando você captura um link de um e-mail ou documento e precisa acessá-lo sem sair da página onde o robô está logado.

18.1. Expressões em linha de comando

Funcionam como "atalhos" ou "tags" que podem ser inseridas em quase qualquer campo (como URLs, nomes de arquivos ou corpos de e-mail) para que o sistema substitua o texto pelo valor real no momento da execução, essas expressões permitem que o robô seja autoconsciente do tempo, do espaço (pastas) e dos dados que estão trafegando na memória.

Tags

- **{{DATE}}**: Retorna a data atual no formato **yyyy-mm-dd** (ex: 2026_03_26). Ideal para criar pastas diárias de backup.
- **{{TIME}}**: Retorna a hora atual no formato **hh_nn_ss**. Útil para dar nomes únicos a arquivos baixados no mesmo dia.
- **{{PATH}}**: Retorna o caminho completo da pasta onde o executável do MyAI está instalado. Essencial para referenciar arquivos internos sem fixar o caminho "C:\MyAI".
- **{{ReaderVariable[</IdAgente/></Variavel/>]}}**: Busca o valor de uma variável. Ex: Usar o nome de um cliente capturado em um site para compor o assunto de um e-mail, poder usar **<< e >>**.
- **{{ReadRepeaters[</IdAgente/></Sequencia/>]}}**: Acessa diretamente um campo do cadastro de repetidores em processamento. Ex: Pegar o CNPJ que está na sequência 1 para preencher um campo de busca.
- **{{MSGResult}}**: Retorna qual botão foi clicado em uma caixa de mensagem (**Message**). Permite que o robô saiba se o usuário clicou em "Sim" ou "Não" para decidir o próximo passo.
- **{{PeriodStart[</Id_agente/></true/>]}}** Retorna período inicial na configuração do agente, com barras ou sem barras
- **{{PeriodEnd[</Id_agente/></true/>]}}** Retorna período final na configuração do agente com barras ou sem barras
- **{{ReaderDataFrame[</dataframe/></Linha/></Coluna/>]}}** extrai uma informação específica de uma tabela, a contagem de linha e coluna começa da zero (0), O DataFrame deve ter sido carregado anterior, Linha ou Coluna pode ser o nome de uma variável de memória
- **{{WriteDataFrame[</dataframe/></Linha/></Coluna/></Texto/>]}}** permite a escrita precisa de informações em um DataFrame carregado na memória, aceita textos fixos ou tags dinâmicas como **{{sysResultado}}**, Linha ou Coluna pode ser o nome de uma variável de memória

18.2. Expressões em linha de comando

- `{{PeriodMMAAA[</Id_agente/></true/>]}}` Retorna período em formato mês e ano

Exemplo Prático de Uso Combinado

Imagine que você quer salvar um arquivo com o nome da empresa e a data de hoje. No campo de nome do arquivo, você escreveria: `Relatorio_{{readrepeaters[</42/></2/>]}}_{{DATE}}.pdf`

O resultado final seria: `Relatorio_EmpresaABC_2026_03_26.pdf`

19. Copiar ou Colar templates

19.1. Copiar

Permite a portabilidade e a escalabilidade dentro do Ecossistema MyAI. Em vez de reconstruir cada lógica do zero, você trata a inteligência do robô como um "pacote" de dados que pode ser replicado em outros ambientes ou computadores. É a forma mais rápida de fazer backup e distribuição.



19.2. Colar

Esta função lê o arquivo de template gerado e reconstrói instantaneamente todo o Ecossistema no novo destino.

MyAI encapsula

- **Agentes:** Com todos microsserviços, eventos, pacotes e memória.
- **Microsserviços:** Com o microsserviços atual e todos os eventos.
- **Eventos:** Com o evento atual.
- **Memória:** Com todas as variáveis de sistema definidas (ex: `sysResultado`).
- **Repetidores:** Com todas empresas cadastradas.

20. Selecionar empresa20. Selecionar

A função **Selecionar** atua como o gatilho que define qual "empresa" ou item de uma lista o robô deve processar naquele momento. Ela é intrinsecamente ligada ao conceito de **Repetidores e Filas de Dados**.



Objetivo :

Carregar os dados de uma empresa específica (CNPJ, Razão Social, Credenciais) para a memória ativa do Agente, **Vínculo com Repetidor**: Ao clicar em Selecionar, o sistema percorre a lista de empresas cadastradas no repetidor. Se houver 10 empresas na fila, o robô executa o agente 1 vezes.

21. Usuários21. Usuários

Segurança e o controle de acesso ao ambiente de automação, o Ecosystema possui um módulo dedicado à gestão de permissões. Este processo é centralizado na tela de **Usuários**, onde o administrador define quem tem acesso ao Ecosystema.



Objetivo :

Esta interface permite o controle total sobre as credenciais de acesso ao aplicativo, garantindo que as automações críticas não sejam alteradas por pessoal não autorizado.



O ícone do **cadeado** é o mecanismo de segurança que trava ou libera a criação de novos registros no sistema.

1. **Estado Travado:** Impede acesso de usuário ao aplicativo.
2. **Estado Liberado:** o usuário tem permissão de acesso ao aplicativo.

22. Pacotes + Repetidores + Memória + Cert + Conf

22.1. Pacotes

Pacotes é os módulos necessários para o agente funcionar



22.2. Repetidores

É uma lista de itens a ser repetida em cada agente, é uma lista de empresas.



22.3. Memória

Gerenciamento de **variáveis de memória**.



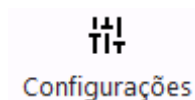
22.4. Certificados

Mapeamento de **certificados digitais** instalados na máquina.



22.5. Configurações

Informações básicas do aplicativo.



23. Terminal de comandos23. Terminal de comandos

O terminal é o ambiente de testes e execução rápida do Ecosystema, onde você pode interagir diretamente com a lógica programada. Você pode digitar comandos em python exemplo `print("Olá MyAI")` e clicar em Executar, você verá do seu lado direito o retorno.



- **Botão Limpar:** Remove todo o histórico de logs e códigos digitados no terminal, deixando a área de visualização pronta para uma nova análise, **ficando na memória os comandos anteriores digitados.**
- **Botão Executar:** Dispara o script em Python que foi digitado ou carregado na área de edição do terminal. É ideal para validar pequenas lógicas de cálculo ou manipulação de strings antes de inseri-las em um eventos.
- **Agentes:** Executa de forma imediata o **Agente selecionado**. Útil para rodar a automação completa após uma manutenção rápida.
- **Microserviços:** Executa de forma imediata o **Microserviços selecionado**.
- **Eventos:** Executa de forma imediata o **Evento selecionado**.

24. Play Agentes

Agentes autônomos é o "motor" invisível que mantém o robô operando 24/7, verificando prazos e gerenciando a fila de empresas.



24.2. Stop Agentes

Indica que o processamento está ativo e monitorando as tarefas.



Quando um robô utiliza funções de **Interface de Usuário (UI)**, como simulação de teclado e movimentos de mouse, ele assume o controle do hardware físico. Se um humano tentar usar o computador ao mesmo tempo, ocorrerá um conflito de comandos, levando a erros no processo ou no preenchimento de dados.

Dicas para Automações que "Ficam Visíveis"

Se o seu Agente precisa obrigatoriamente usar comandos de mouse e teclado:

- **MinimizeBrowser:** Evite usar o `minimizebrowser` se a automação depender de cliques visuais em elementos que só aparecem quando a janela está em foco.
- **MaximizeBrowser:** Use sempre no início do agendamento noturno para garantir que o layout esteja correto para os cliques do mouse.
- **Estação Dedicada:** O ideal é que o Ecossistema rode em uma **Máquina Virtual (VM)** ou em um servidor dedicado. Assim, o robô pode "mexer o mouse" à vontade sem interromper o trabalho de ninguém. **Não use o seu Mouse Físico DENTRO da VM enquanto o robô trabalha:** Se você abrir a janela da VM para "dar uma espiadinha" e mexer o mouse lá dentro, aí sim você vai brigar com o robô pelo controle daquele cursor específico.
- Pode usar um **Servidor Virtual Privado (VPS)** para seu Ecossistema, a VPS é a **solução definitiva de "Esqueça e Deixe Rodar"**.

O Conceito de Isolamento de Hardware

Quando você roda uma VM (seja no VirtualBox, VMware ou Hyper-V), o software cria uma camada de abstração. **Docker não possui um cursor de mouse ou teclado virtual por isso não deve usar.**

25. Play Telegram

Controle Remoto permite que você monitore o ciclo de execução dos agentes em tempo real. Se o robô estiver processando a lista de empresas (N vezes) e encontrar um erro ou finalizar a tarefa, ele enviará uma mensagem diretamente para o seu celular.



25.2. Stop Telegram

Indica que o processamento está ativo e monitorando comandos no Telegram.



Uma regra obrigatória da API Telegram é que o app Telegram envia uma mensagem e o MyAI responde, não funciona se o MyAI enviar uma mensagem para o app Telegram responder.

Configuração do Robô (Passo a Passo)

Para ativar essa função, siga o procedimento técnico para obter sua **API Token**:

1. **Abertura**: Instale e abra o aplicativo Telegram em seu smartphone ou desktop.
2. **Busca**: Procure pelo usuário oficial **@BotFather** (o "pai" de todos os bots).
3. **Criação**: Digite o comando `/newbot`.
4. **Identificação**: Escolha um **Nome** de exibição (ex: `MonitorEcosystemabot`).
5. **Obtenção do Token**: Após a criação, o BotFather enviará uma mensagem contendo uma sequência de números e letras (a **API Token**). Caso já tenha um bot, use `/mybots`, selecione o robô e clique em **API Token**.

□ Vinculação no Ecosystema

Com o Token em mãos, você deve registrá-lo no "cérebro" do aplicativo para que a comunicação seja estabelecida:

- **Acesse Configurações**: No menu principal do Ecosystema, clique no ícone de **Configurações**.

Id Autorização Telegram: * aceita todos usuários ou separar por barras (/) cada Id.

OBS. não se esqueça de acessar o seu bot criado.

26.1. Configuração de DataFrame

Configuração de DataFrame

Análise de dados e transformação de arquivos brutos em tabelas estruturadas prontas para a automação. mapa que diz ao robô de onde os dados vêm e para onde eles devem ir após o processamento.

Estrutura

Para carregar uma tabela na memória, lista de parâmetros:

1. **Instância** : É o nome único do seu DataFrame na memória. Use este nome quando for chamar as funções `ReaderDataFrame` ou `WriteDataFrame`
2. **Tipo** : Define o driver de leitura. O sistema suporta:
 - **Arquivos**: Excel (.xlsx) e CSV.
 - **Bancos de Dados SQL**: Suporte nativo para Firebird, Interbase, MariaDB, MSSQL, MySQL, Oracle, PostgreSQL, SQLite e Sybase.
3. **Parâmetros Específicos**:
 - **Excel**: Defina qual **Aba** (Sheet) ler.
 - **CSV**: Defina o **Delimitador** (ex: vírgula ou ponto e vírgula).
 - **SQL**: Insira a **Cláusula WHERE** para filtrar os dados diretamente na fonte.
4. **Localização**: O caminho completo do arquivo para Excel/CSV ou o nome da tabela para bancos SQL.
5. **Pular Linhas (Skip Rows)**: Útil para planilhas que possuem logotipos ou títulos nas primeiras linhas antes do cabeçalho real.

Mapeamento

O robô reside na capacidade de converter dados entre diferentes formatos e sistemas:

- **Colunas de Entrada**: É o mapeamento técnico de todas as colunas que o robô encontrará na origem (tabela original).
- **Colunas de Saída**: Define para onde os dados irão. Aqui você vincula a coluna "A" da entrada à coluna "B" do destino.
 - *Nota Automática*: Se você estiver apenas convertendo de **Excel para CSV** (ou vice-versa), o sistema mapeia as colunas automaticamente, dispensando a configuração manual de saída.

Exportação e filtros de dados para tabelas

O módulo de **Tabela** define as regras de exportação direta do DataFrame para um banco de dados destino:

Conversão de valores, o formato padrão é . e não , ex. 999,999.99 para o formato brasileiro deve fazer a troca "." por "," ex. `searchdata(</select_data_856/></1/></./></./></S/>)`

26.2. Comandos de um DataFrame

Comandos de um DataFrame

searchdata (Pesquisa e Extração Inteligente)

Sintaxe: `searchdata(</DF/></Coluna/></Pesquisar/></Substituir/></Texto/></Modo/>)`

- Retornando posição:

```
</select_data_809/></0/></9876543212345/><///><///></P/>
```

- Retornando Texto esquerda

```
<///><///></9876543212345/><///></mmm 9876543212345 yyy/></TL/>
```

- Retornando Texto direita

```
<///><///></9876543212345/><///></mmm 9876543212345 yyy/></TR/>
```

- Deixa apenas numeros na coluna

```
</select_data_809/></0/><///><///><///></N/>
```

- Deixa apenas 0123456789., na colua

```
</select_data_809/></0/></mmmmmm987,345/><///><///></M/>
```

- Substitui texto na coluna

```
</select_data_809/></0/></de/></de milho/><///></S/>
```

- Número de linhas

```
</select_data_809/><///><///><///><///></L/>
```

- Número de colunas

```
</select_data_809/><///><///><///><///></C/>
```

- Exclui coluna 0

```
</select_data_809/></0/><///><///><///></E/>
```

- Remove os nomes estão "quebrados" (espaços invisíveis)

```
</select_data_809/><///><///><///><///></Q/>
```

- Renomear cabeçalhos específicos

```
searchdata(</select_data_514/></'Fantasia': 'A1', 'CNPJ/CPF': 'AB'/><///><///><///></O/>)
```

- Filtro texto na coluna

```
</select_data_809/></0/><!/=></Enviado/><///></F/>
```

Sintaxe de expressões (> : Maior, < : Menor,
 >= : Maior ou igual, <= : Menor ou igual,
 = : Igual, != : Diferente)

26.2.1. Comandos de um DataFrame

- Limite de total de linhas retornadas:

```
</select_data_809/></80/><///><///><///></l/>
```

26.3. Comandos de um DataFrame

Comandos de um DataFrame

Conexões de Banco de Dados

- **databaseentryconnection**: Abre a conexão de **entrada** para importar dados para o DataFrame (usando os protocolos como MySQL, PostgreSQL, etc).
- **databaseexitconnection**: Abre a conexão de **saída** para onde os dados processados serão enviados.
 - *Parâmetros*: </Protocolo/></Database/></Host/></Port/></User/></Pass/>
Protocolo = (firebird, interbase, mariadb, mssql, mysql, oracle, postgresql, sqlite, sybase, csv, excel)
- **closeconnection**: Fecha a conexão (**entry** ou **exit**) para liberar memória e evitar travamentos no banco.

exportdataframe (Salvar Resultados)

Exporta o conteúdo da memória para um arquivo físico ou tabela de banco de dados.

Sintaxe:

exportdataframe(</DF/></Col_Entrada/></Tabela_Destino/></Col_Saida/></Filtro/></Expressao/></Conteudo/>)

- Permite mapear quais colunas da memória vão para quais colunas do banco.
- Possui motor de **filtro interno**: você pode exportar apenas linhas onde a "Coluna X" tenha o "Conteúdo Y".

26.4. Comandos de um DataFrame

Comandos de um DataFrame

Manipulação de Estrutura

- **dropduplicates**: Remove linhas repetidas. Pode-se combinar várias colunas para definir o que é uma duplicidade única.

Sintaxe: `dropduplicates(</DF/></Coluna1/></Coluna2/>...)`

- **variablereplace**: Pega o resultado de um processamento e o salva em uma Variável de Memória (`?Variavel?`), permitindo que outros microserviços usem esse dado.
- **readerdataframe** / **writerdataframe**: Como vimos, realizam a leitura e escrita pontual em células específicas (Linha/Coluna).
- **machinelearning** : Correlação de dados para descobrir padrões escondidos
cor = Correlações de Dados
- **groupby** : Agrupar dados
sum = Soma
count = Conta
mean = Média
max = Valor máximo
min = Valor mínimo
- **columnsdataframe** : Você poder criar os títulos para uma dataframe

26.5. Escrever em um DataFrame26.5.

Escrever em um DataFrame

Função : writedataframe

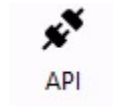
Permite a escrita precisa de informações em um DataFrame carregado na memória, aceita textos fixos ou tags dinâmicas como {{sysResultado}}, Linha ou Coluna pode ser o nome de uma variável de memória.

Parâmetros;

- 1) Dataframe nome da instância
- 2) Linha a ser escrita
- 3) Coluna a ser escrita
- 4) Texto a ser escrito

27. API (Interface de Programação de Aplicações)

Funciona como uma "ponte" ou um tradutor que permite que dois softwares diferentes conversem entre si. .



27.1 Central de inteligência multi-modelo

- **IP** : url endpoint
- **Porta** : porta de comunicação
- **Autorização**: ex. Bearer <<tokenapi>> substituirá automaticamente pelo token
- **Método** : post ou get
- **Corpo** : Sintaxe de envio da mensagem, abaixo ex. IA Grok;
{ "model": "grok-beta", "messages": [{ "role": "system", "content": "<<systemapi>>
<<skillsapi>>" }, { "role": "user", "content": "<<messageapi>>" }], "temperature": 0.7, "stream": false }
- **Instruções gerais System** : Aqui escreve o comportamento base (ex: "Você é um assistente prestativo"). No JSON, isso será injetado em <<systemapi>>.
- **Habilidades Skills** : Aqui coloca conhecimentos específicos ou regras de negócio. No JSON, isso será injetado em <<skillsapi>>.
- **Retorno**: `choices[0].message.content` se deixar em branco traz tudo o retorno para posterior configuração

Sintaxe de TAG

system = <<systemapi>> : Instruções gerais (Você é um professor de matemática, Especializado em programação)

skills = <<skillsapi>> : Habilidades

input = <<messageapi>> : Mensagem

to = <<numberapi>> : Número

file = <<fileapi>> : Arquivo

url = <<ipapi>> : URL da API

port = <<portapi>> : Porta de comunicação

token = <<tokenapi>> : Token

clientid = <<clientidapi>> : Identificação do cliente